

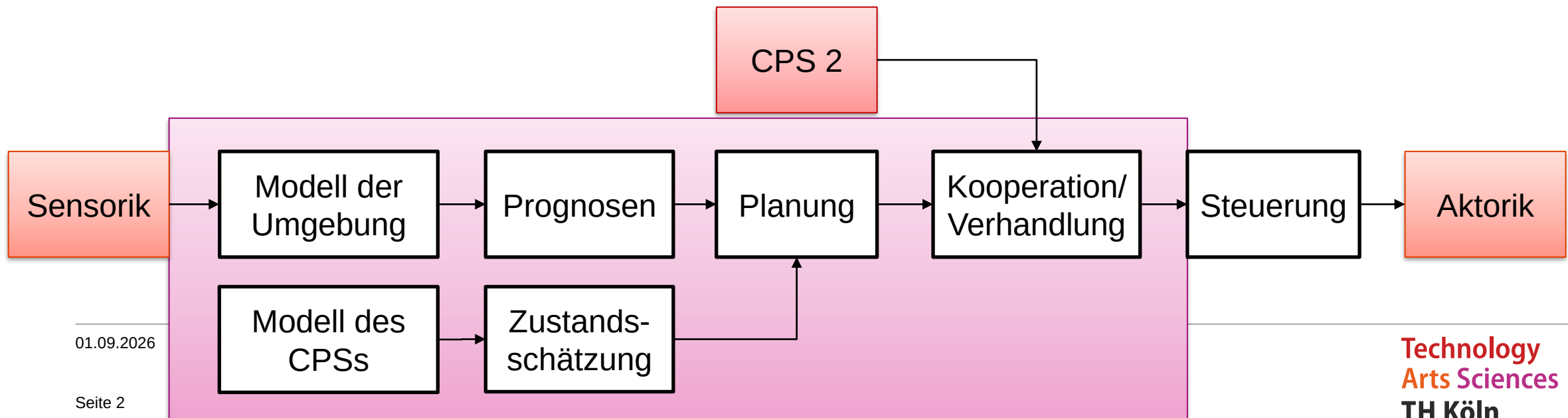


Cyber-Physische Systeme – WS 26/27

Gelesen von Prof. Dr. Daniel Gaida

Inhalt

- Prognose und Planung



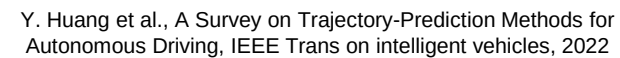
Inhalt

Prognose:

- Prognose mit Deep Learning Modellen

Planung:

- Bewegungsplanung
 - Configuration Space (C-Space)
 - Approximation des C_{free} -Space durch einen Graph
 - Algorithmus von Dijkstra und A* für Suche des kürzesten Pfades auf dem Graph
 - Probabilistic Roadmap und Rapidly Exploring Random Tree für bessere Graph Erstellung
- Aufgabenplanung
 - „Gehe zur großen weißen Couch“



Prognose beim autonomen Fahren

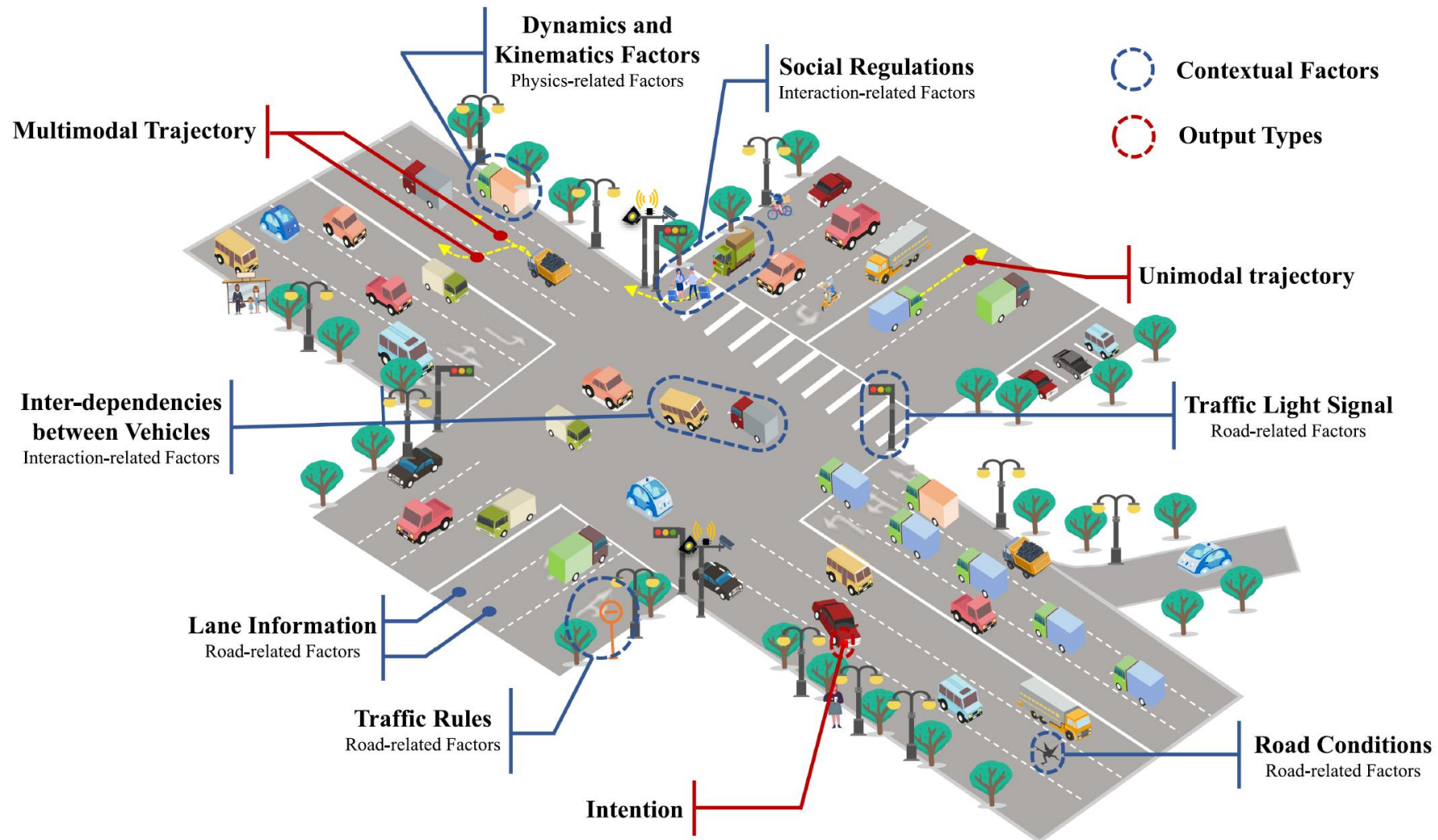
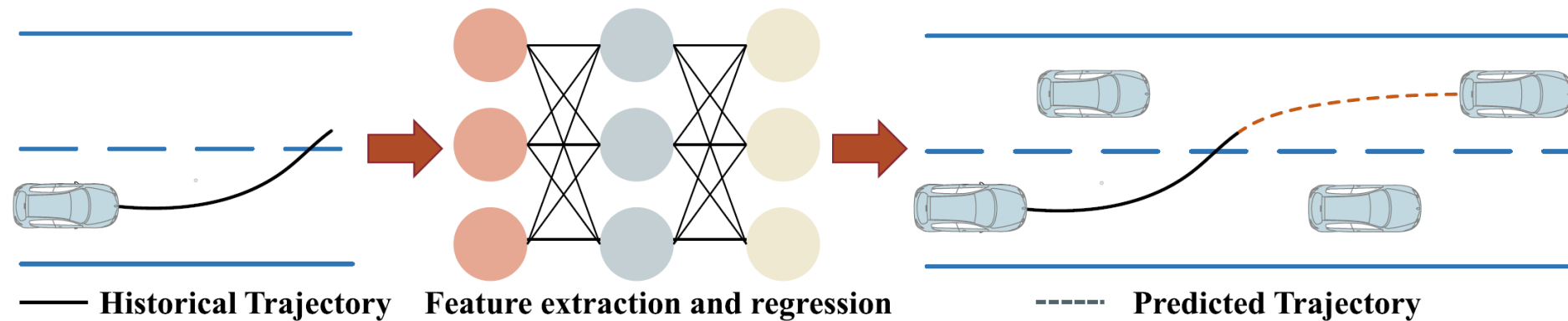


Fig. 3. The input and output factors of trajectory prediction.

Y. Huang et al., A Survey on Trajectory-Prediction Methods for Autonomous Driving, IEEE Trans on intelligent vehicles, 2022

Prognose mit Deep Learning Modellen

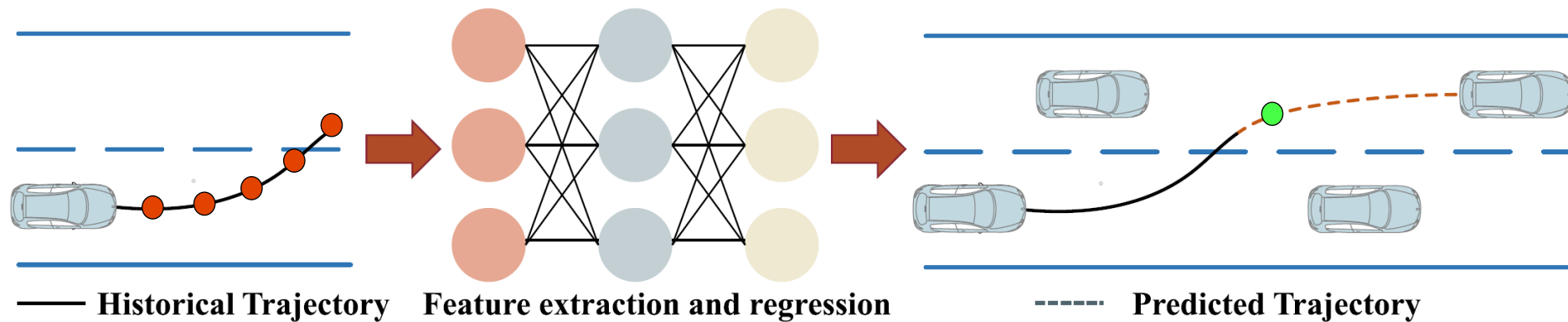
- Sie nehmen die Daten der letzten paar Sekunden (oder Minuten, je nachdem) und geben diese in das Modell hinein
- Sie erhalten eine Prognose



Prognose mit Deep Learning Modellen – statisches Modell

- Sie nehmen die Daten der letzten paar Sekunden (oder Minuten, je nachdem) und geben diese in das Modell hinein
- Sie erhalten eine Prognose für das nächste Zeitintervall

$u_{t-4} \ u_{t-3} \ u_{t-2} \ u_{t-1} \ u_t \rightarrow \text{Modell} \rightarrow y_{t+1}$ (u: Eingangsdaten (x,y), y: Ausgangsdaten)



5 (x,y) Koordinaten $\rightarrow$ 10 Eingangsneuronen

2 Ausgangsneuronen (x,y)

Prognose mit Deep Learning Modellen – statisches Modell

- Sie nehmen die Daten der letzten paar Sekunden (oder Minuten, je nachdem) und geben diese in das Modell hinein
- Sie erhalten eine Prognose für das nächste Zeitintervall
- Um weiter in die Zukunft zu schauen:
 - Sie nehmen die Prognose und die Daten von vorhin (ohne dem ältesten Datenpunkt) und geben diese Daten erneut in das Modell
 - ...

$u_{t-4} \ u_{t-3} \ u_{t-2} \ u_{t-1} \ u_t \rightarrow \text{Modell} \rightarrow y_{t+1}$ (u: Eingangsdaten, y: Ausgangsdaten)

$u_{t-3} \ u_{t-2} \ u_{t-1} \ u_t \ y_{t+1} \rightarrow \text{Modell} \rightarrow y_{t+2}$

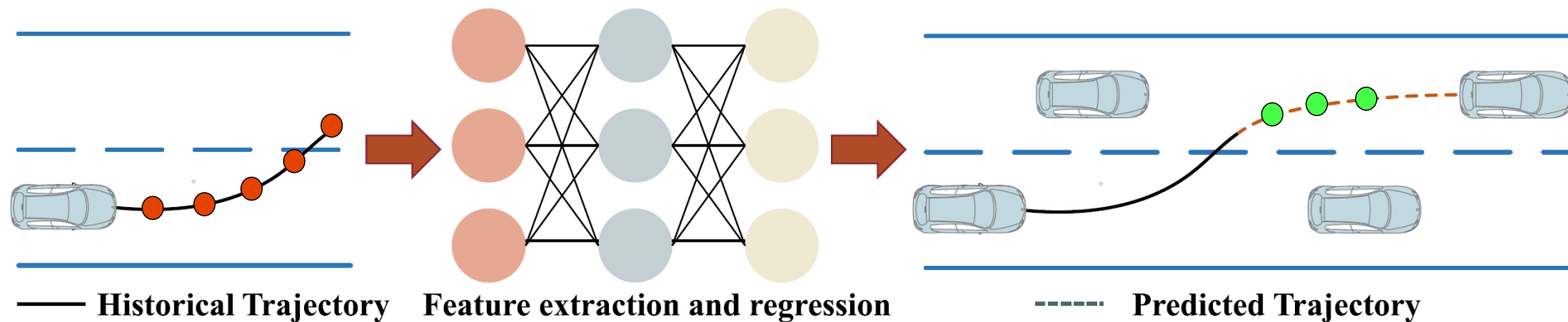
$u_{t-2} \ u_{t-1} \ u_t \ y_{t+1} \ y_{t+2} \rightarrow \text{Modell} \rightarrow y_{t+3}$

Prognose mit Deep Learning Modellen – statisches Modell

$u_{t-4} \ u_{t-3} \ u_{t-2} \ u_{t-1} \ u_t \rightarrow \text{Modell} \rightarrow y_{t+1}$ (u: Eingangsdaten, y: Ausgangsdaten)

$u_{t-3} \ u_{t-2} \ u_{t-1} \ u_t \ y_{t+1} \rightarrow \text{Modell} \rightarrow y_{t+2}$

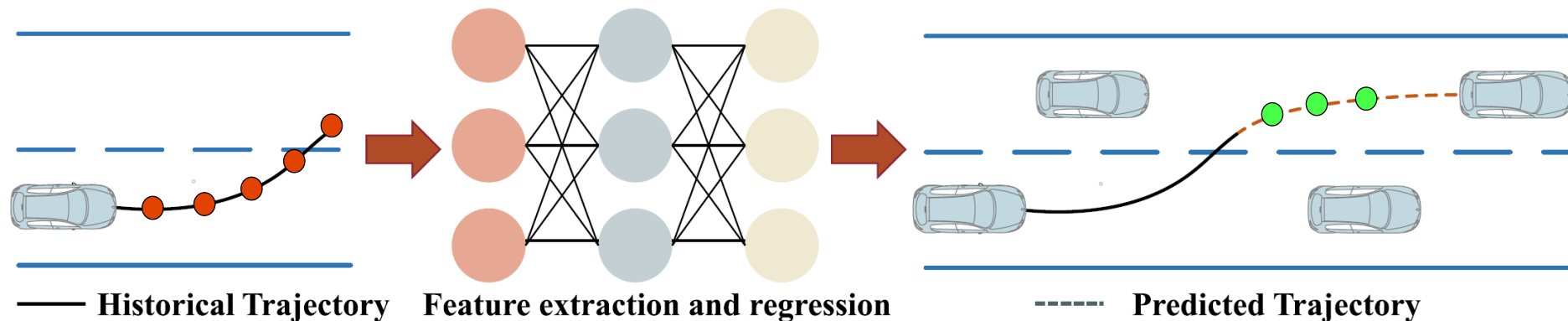
$u_{t-2} \ u_{t-1} \ u_t \ y_{t+1} \ y_{t+2} \rightarrow \text{Modell} \rightarrow y_{t+3}$



Prognose mit Deep Learning Modellen – statisches Modell

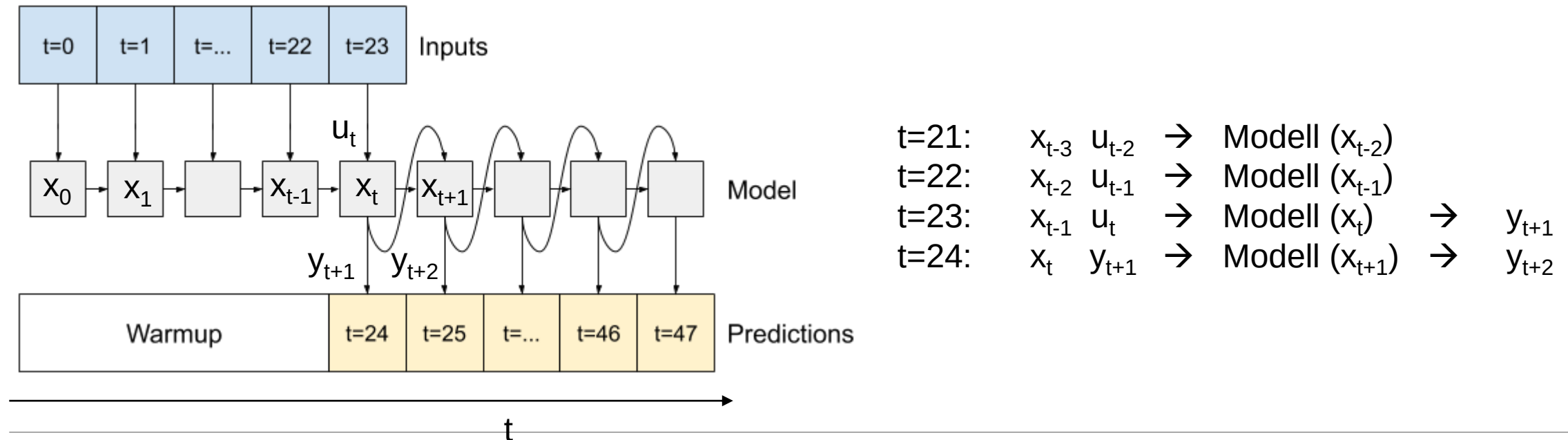
Wie trainiert man das Modell?

- Man zeichnet ganz viele Trajektorien von Fahrzeugen auf: $u_{t-n} \dots u_{t-3} u_{t-2} u_{t-1} u_t$
- Man nutzt die alten Daten als Input und die neuen als Label:
 - Input: $u_{t-n} \dots u_{t-3} u_{t-2} u_{t-1}$ Label: u_t
 - Input: $u_{t-n} \dots u_{t-4} u_{t-3} u_{t-2}$ Label: u_{t-1}
 - ...

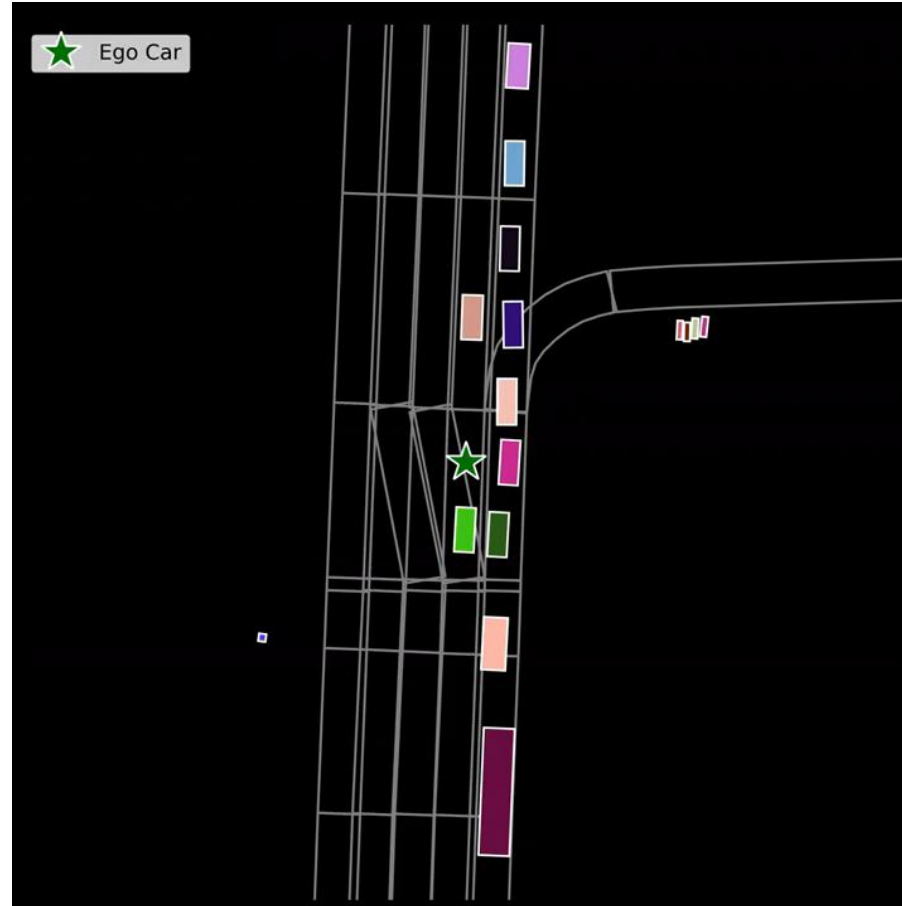


Prognose mit Deep Learning Modellen – dynamisches Modell

- Alternative: Nutzung von rekurrenten neuronalen Netzen
 - Diese Modelle haben einen internen Zustand (Speicher), dessen Werte von den vorherigen Eingangsdaten abhängt
 - Prognose hängt ab von Zustand (x_t) des Modells und Eingangsdaten (u_t)



Prognose des Verhaltens anderer Fahrzeuge

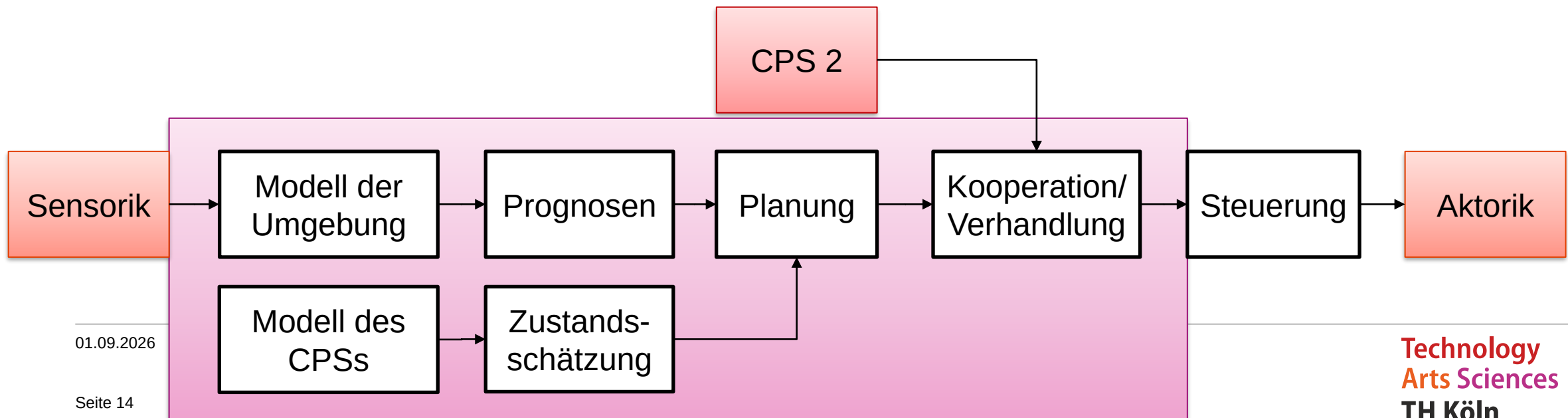


Übung - Prognose

Was für Prognosen muss man im Roboterfußball durchführen?

Inhalt

- Prognose und Planung



Inhalt

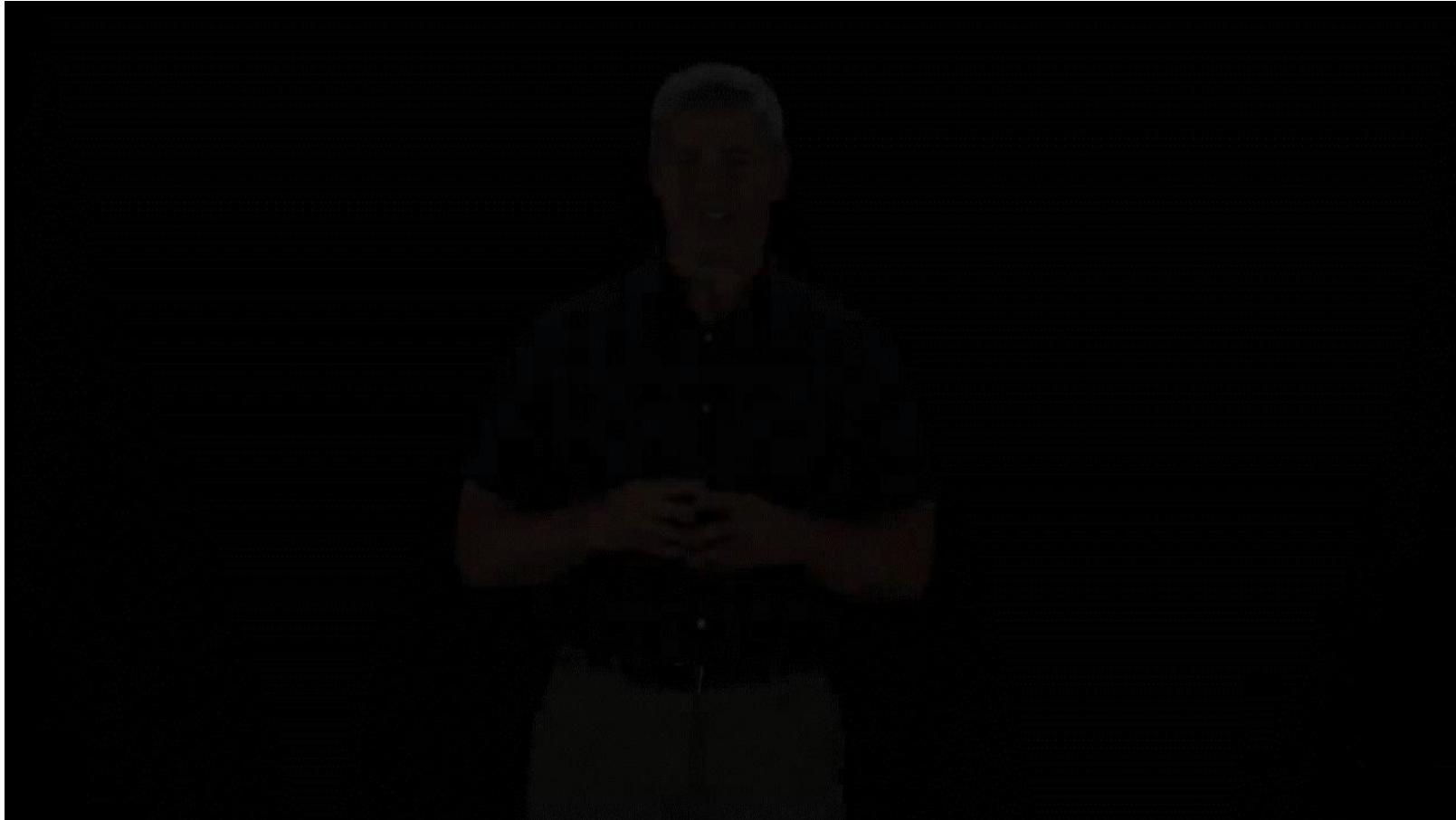
Prognose:

- Prognose mit Deep Learning Modellen

Planung:

- Bewegungsplanung
 - Configuration Space (C-Space)
 - Approximation des C_{free} -Space durch einen Graph
 - Algorithmus von Dijkstra und A* für Suche des kürzesten Pfades auf dem Graph
 - Probabilistic Roadmap und Rapidly Exploring Random Tree für bessere Graph Erstellung
- Aufgabenplanung
 - „Gehe zur großen weißen Couch“

Bewegungsplanung



Bewegungsplanung

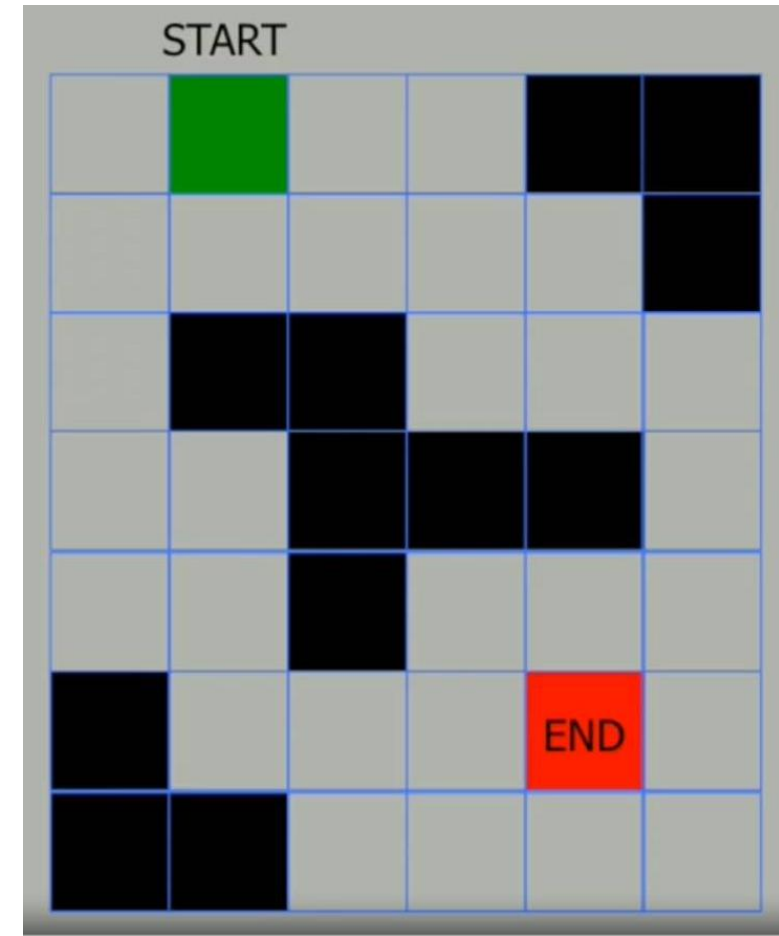
Configuration Space (C-Space)

Der C-Space ist ein Raum, der durch die Wertebereiche der Freiheitsgrade des CPS aufgespannt ist.

Für ein autonom fahrendes System, entspricht der C-Space dem Raum in dem sich das System bewegen kann.

Der C_{free} -Space ist der Teilraum in dem sich das CPS tatsächlich aufhalten darf.

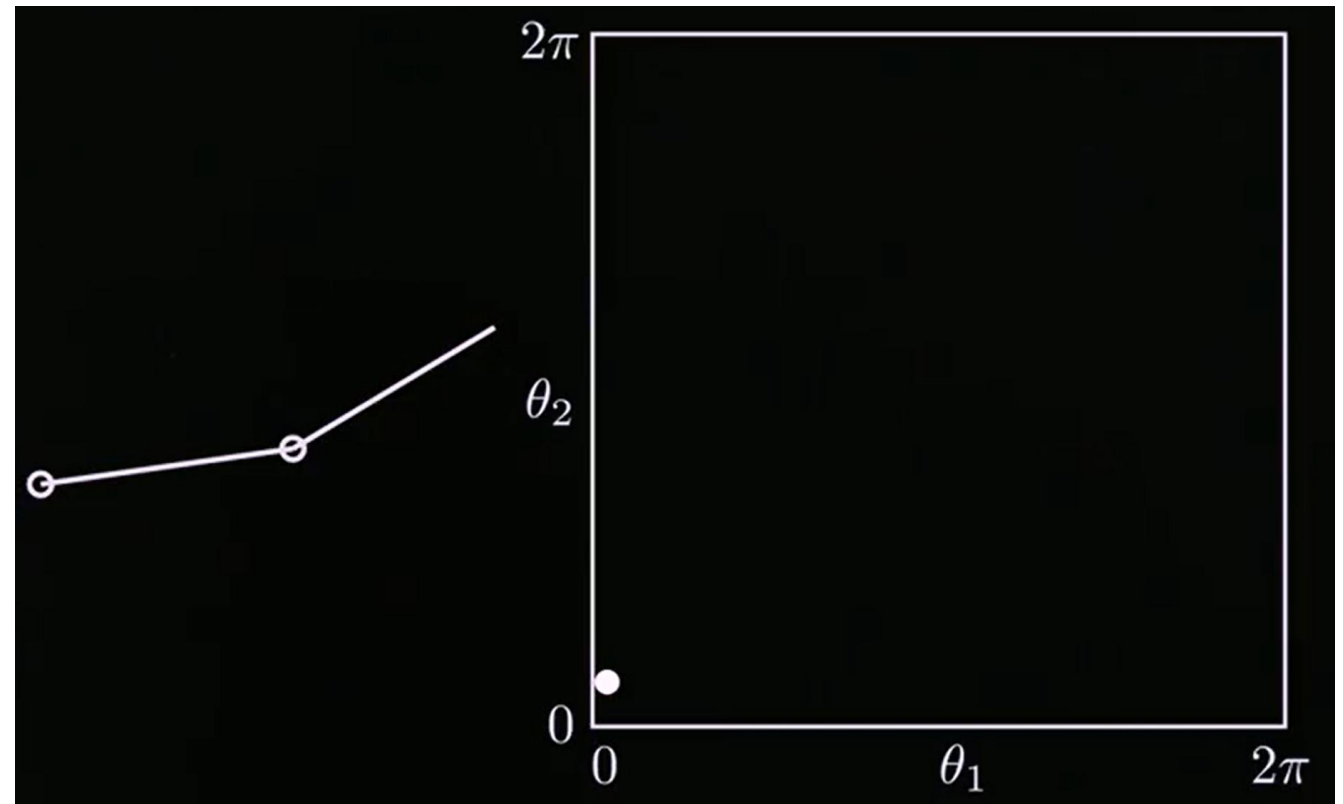
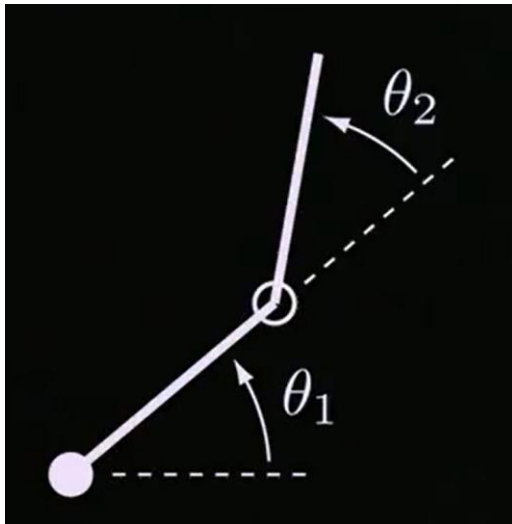
- Bspw. Teile des Raumes, die nicht durch Hindernisse belegt sind



Bewegungsplanung

Configuration Space (C-Space)

Der C-space für einen Roboterarm mit zwei Freiheitsgraden:

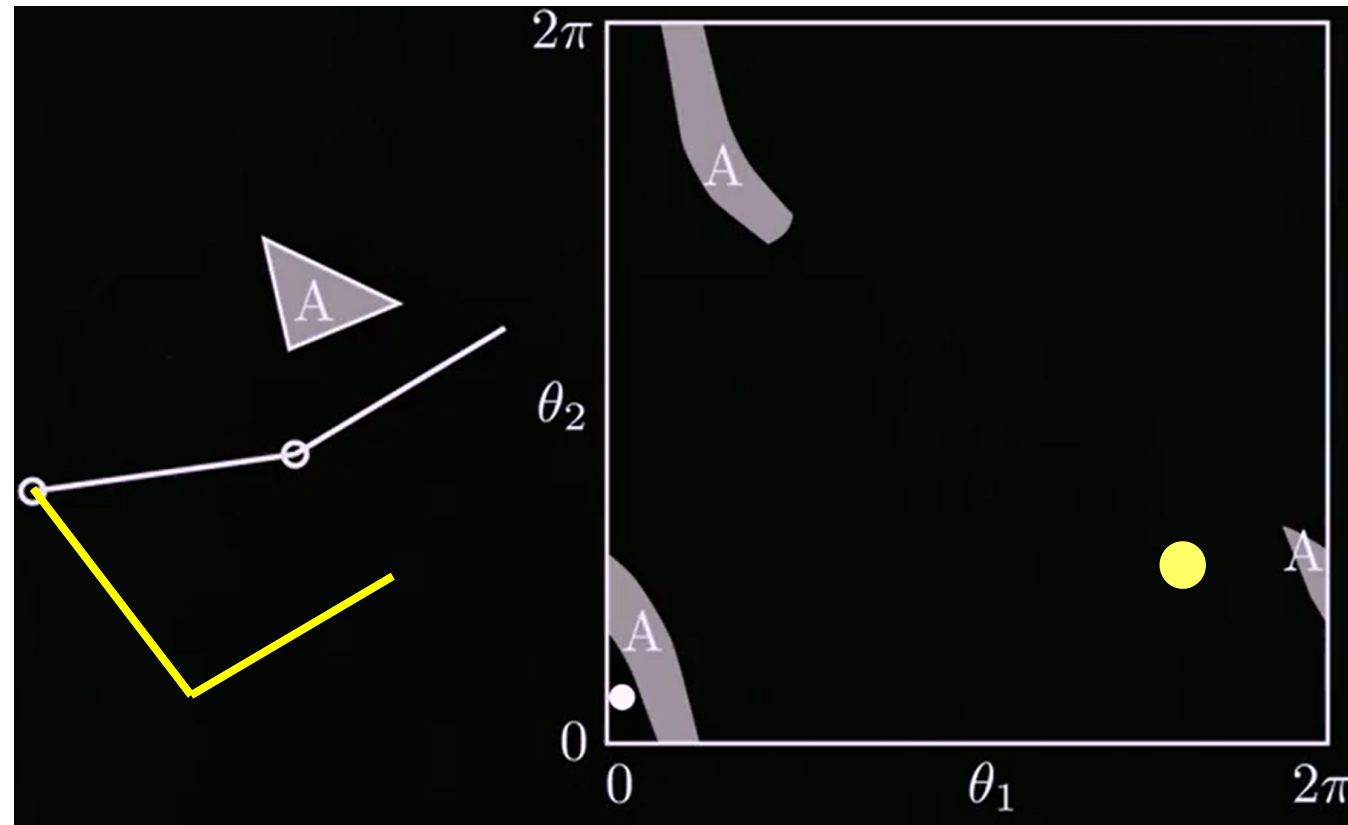


Bewegungsplanung

Configuration Space (C-Space)

<https://www.cs.unc.edu/~jeffi/c-space/robot.shtml>

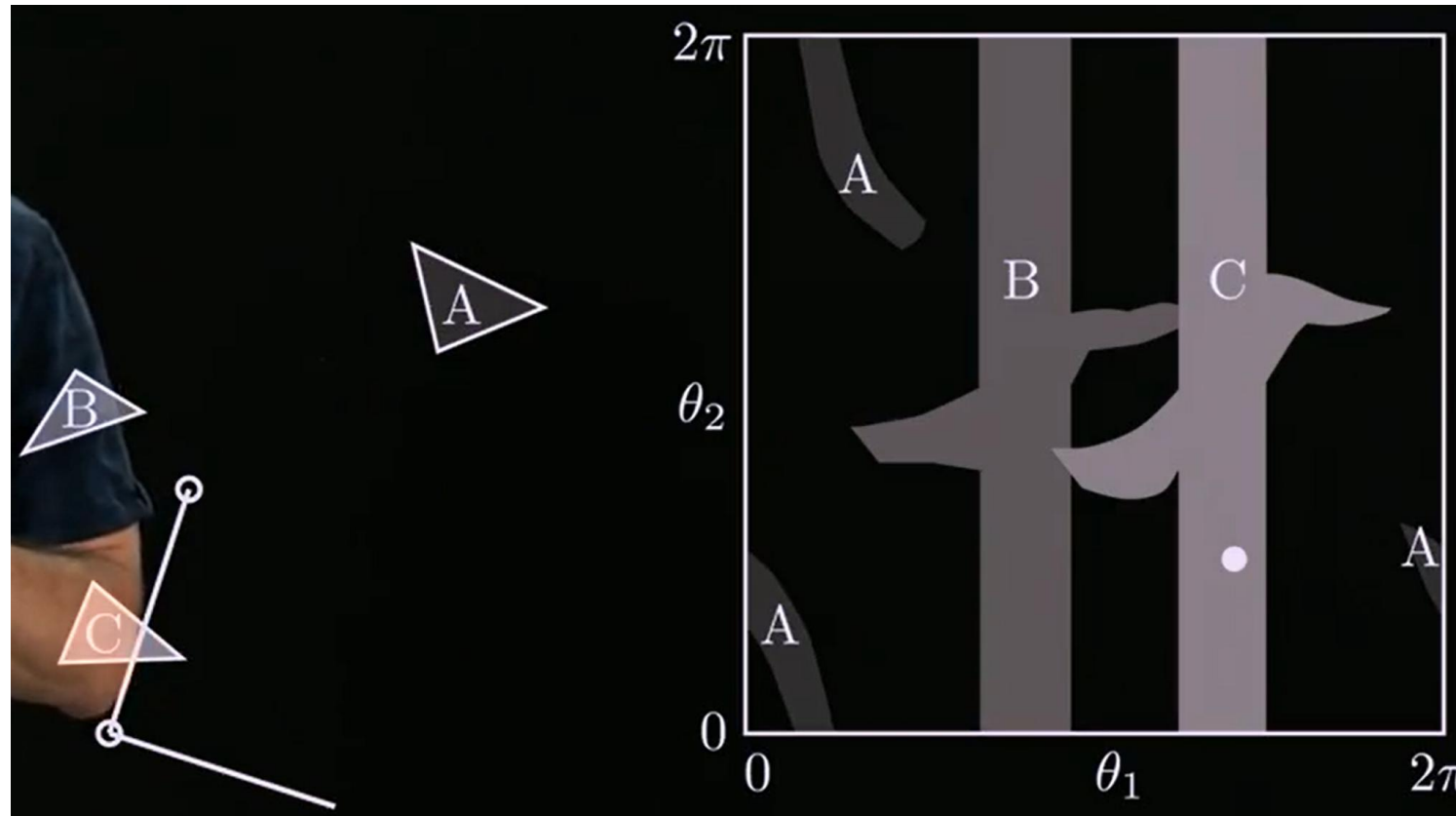
Der C-space für einen Roboterarm mit zwei Freiheitsgraden:



Bewegungsplanung

Configuration Space (C-Space)

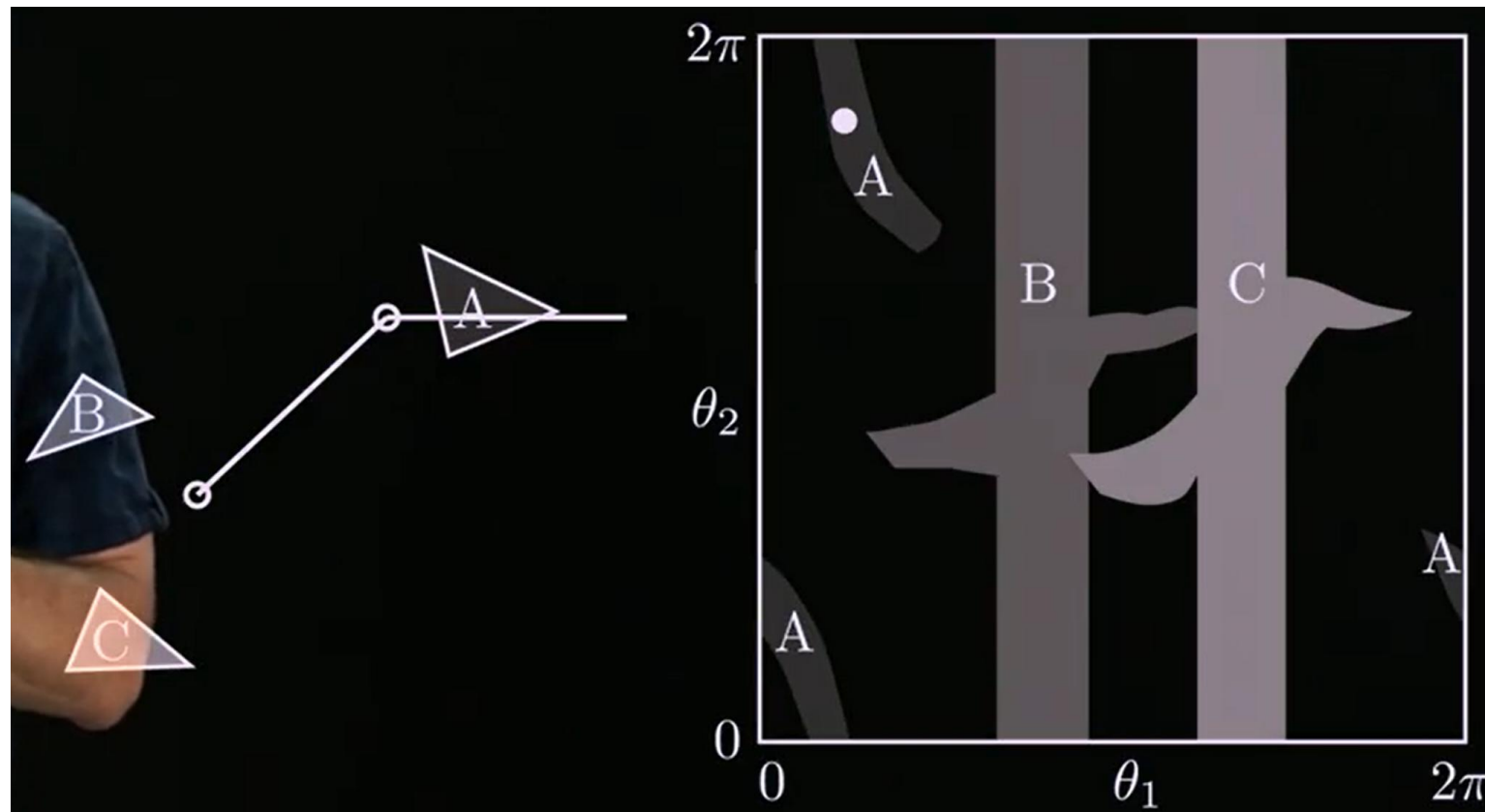
Der C-space für einen Roboterarm mit zwei Freiheitsgraden:



Bewegungsplanung

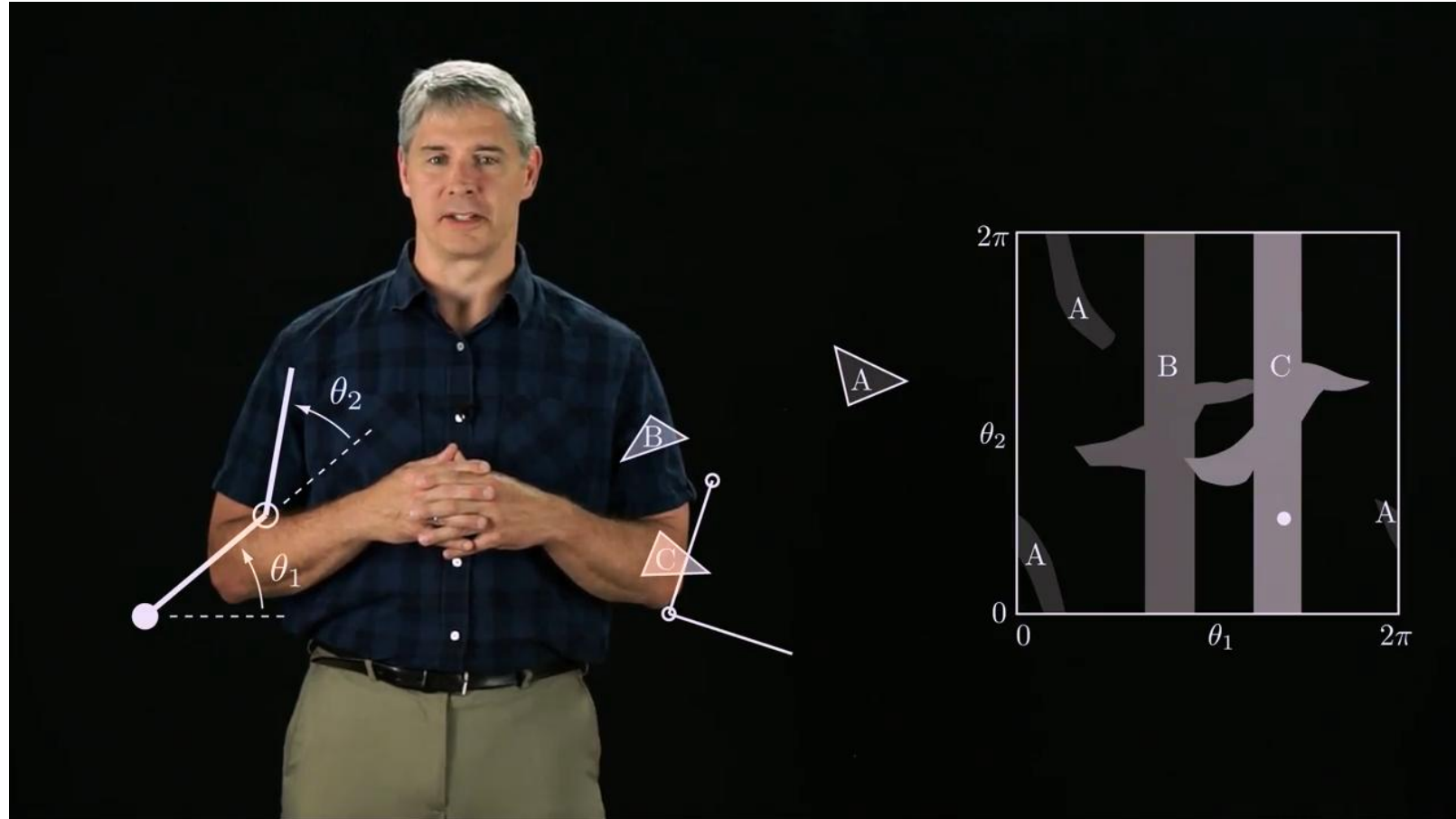
Configuration Space (C-Space)

Der C-space für einen Roboterarm mit zwei Freiheitsgraden:



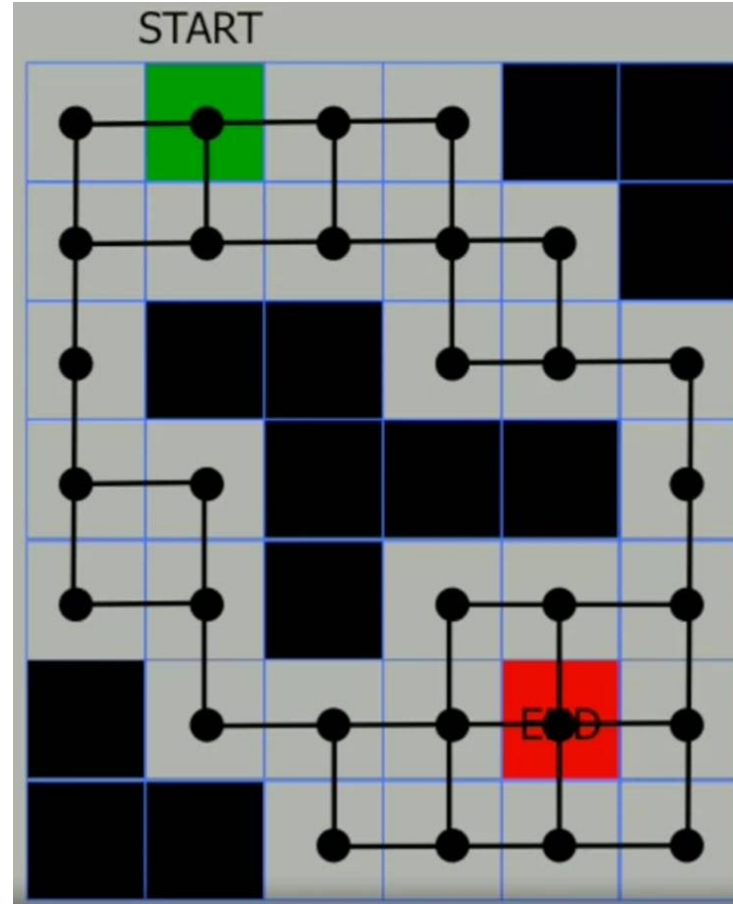
Bewegungsplanung

Configuration Space (C-Space)



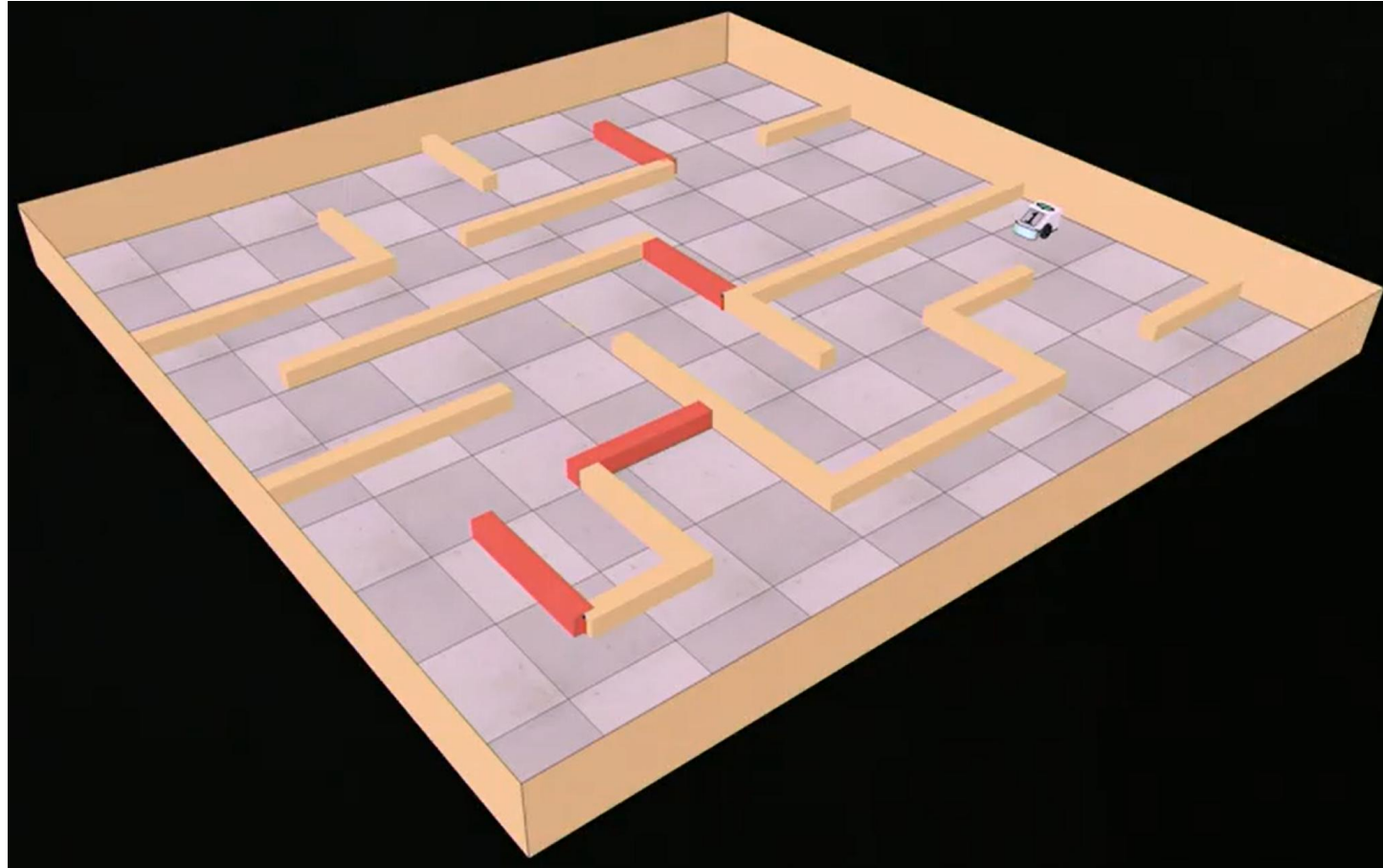
Bewegungsplanung

Approximation des C_{free} -Space durch einen Graph



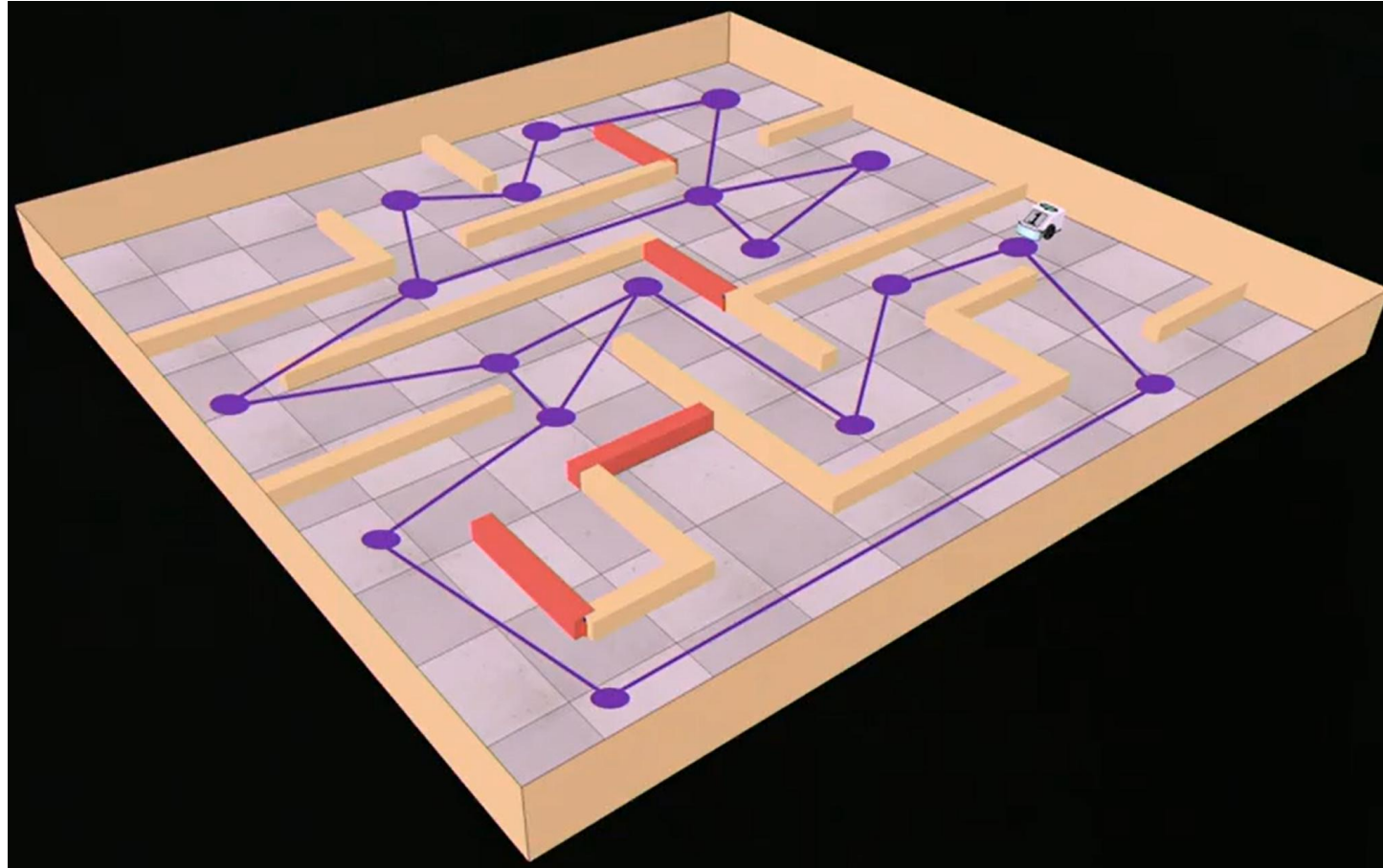
Bewegungsplanung

Approximation des C_{free} -Space durch einen Graph



Bewegungsplanung

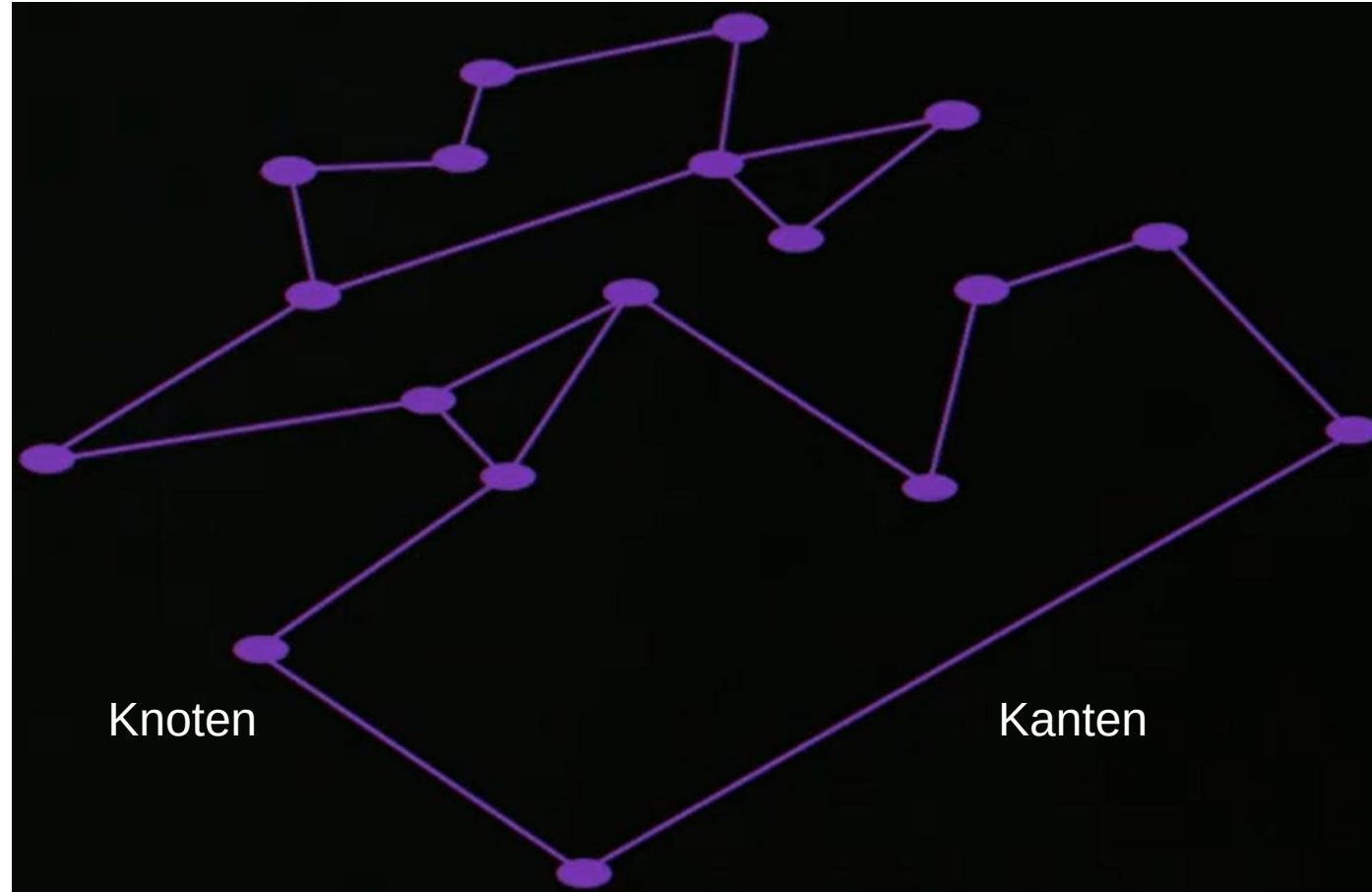
Approximation des C_{free} -Space durch einen Graph



Bewegungsplanung

Approximation des C_{free} -Space durch einen Graph

Graph



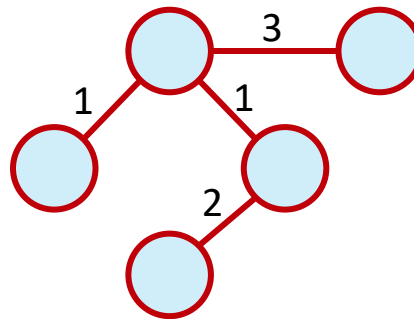
Bewegungsplanung

Gewichteter Graph

An den Kanten stehen Kantengewichte

Bspw. Modellierung von:

- Kosten
- Distanzen
- Zeit
- ...



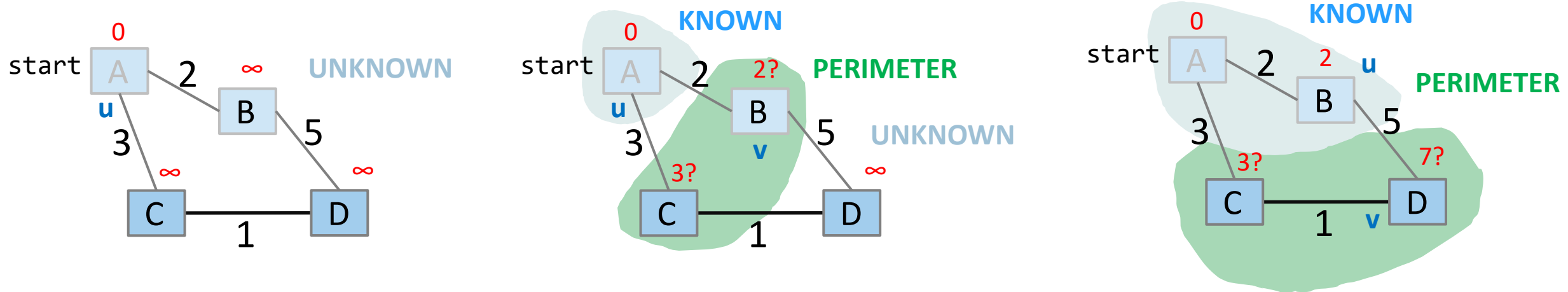
Bewegungsplanung

Suche des kürzesten Pfades für einen Graph

Gewichteter Graph:

- Algorithmus von Dijkstra (Algorithmik Vorlesung der Informatik)
- A* Algorithmus

Dijkstras Algorithmus: Idee



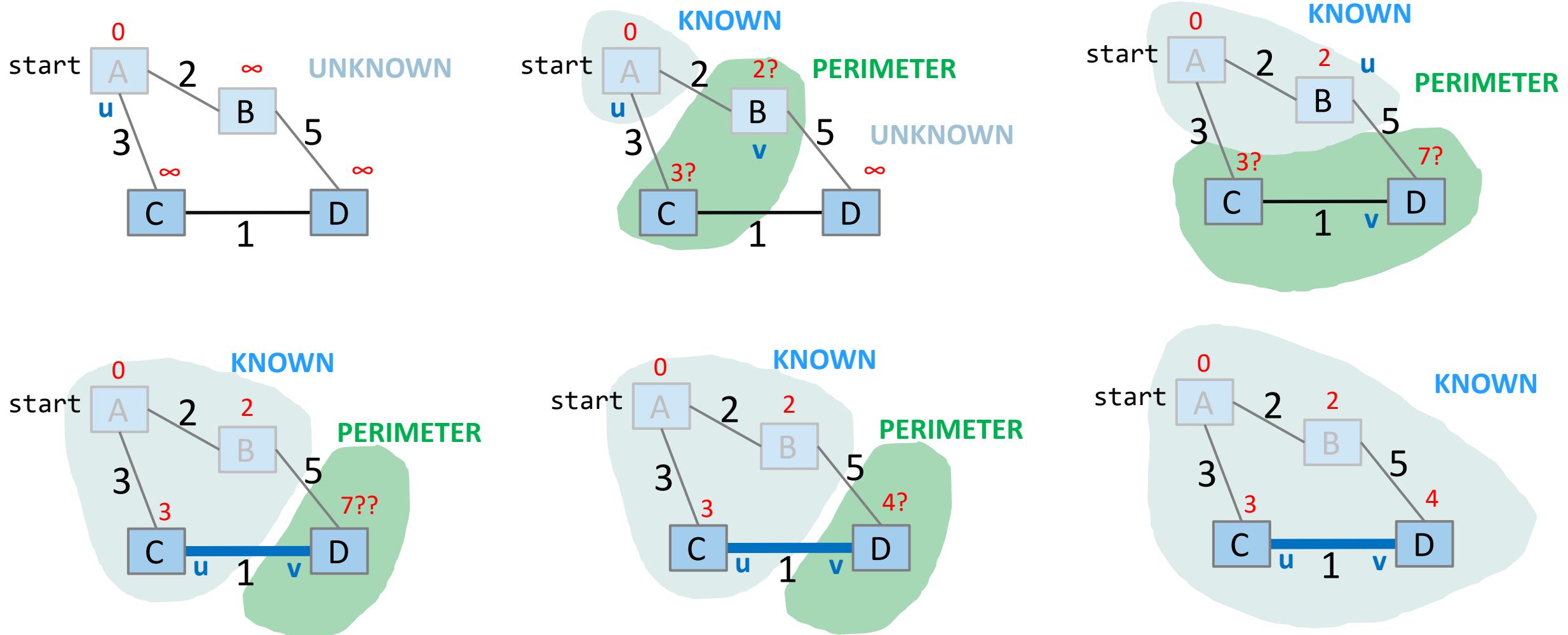
Initialisierung:

- Startknoten hat Abstand 0; alle anderen Knoten haben Abstand ∞

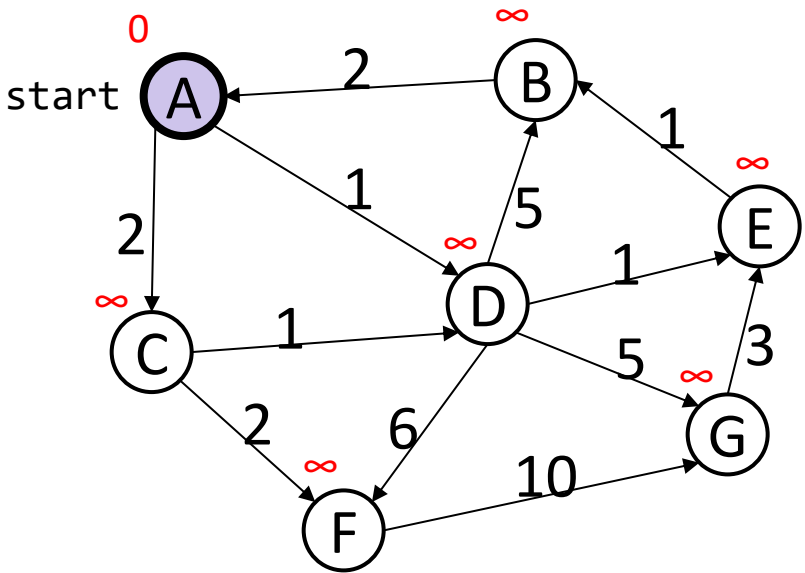
Bei jedem Schritt:

- Nächstgelegenen unbekannten Knoten u auswählen
- Hinzufügen zur „Wolke“ der bekannten Knoten
- Aktualisierung der „bisher besten“ Abstände für Knoten, die Kanten von Knoten u haben

Dijkstras Algorithmus in Bildern



Dijkstras Algorithmus: Übung



Order Added to
Known Set:

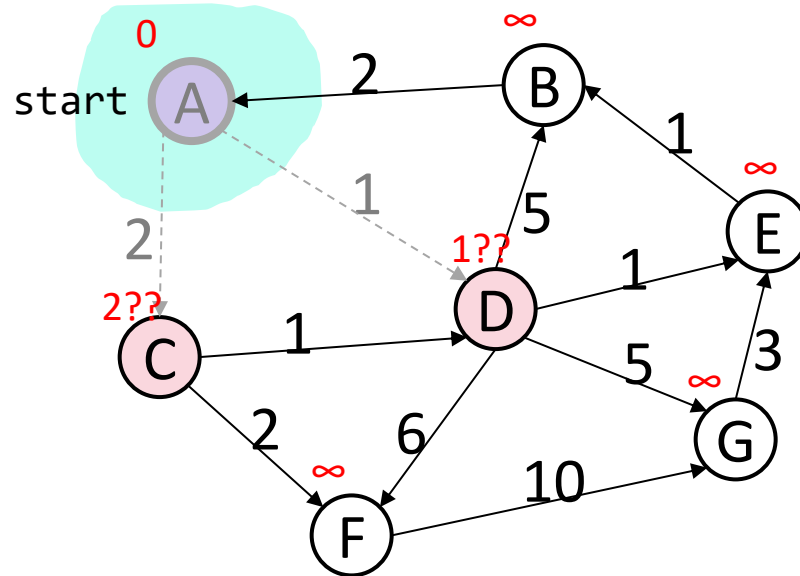
```

while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
    for each edge (u,v) to unknown v with w:
        if distTo.get(u) + w < distTo.get(v):

```

Vertex	Known?	distTo	edgeTo
A		0	
B		∞	
C		∞	
D		∞	
E		∞	
F		∞	
G		∞	

Dijkstras Algorithmus: Übung

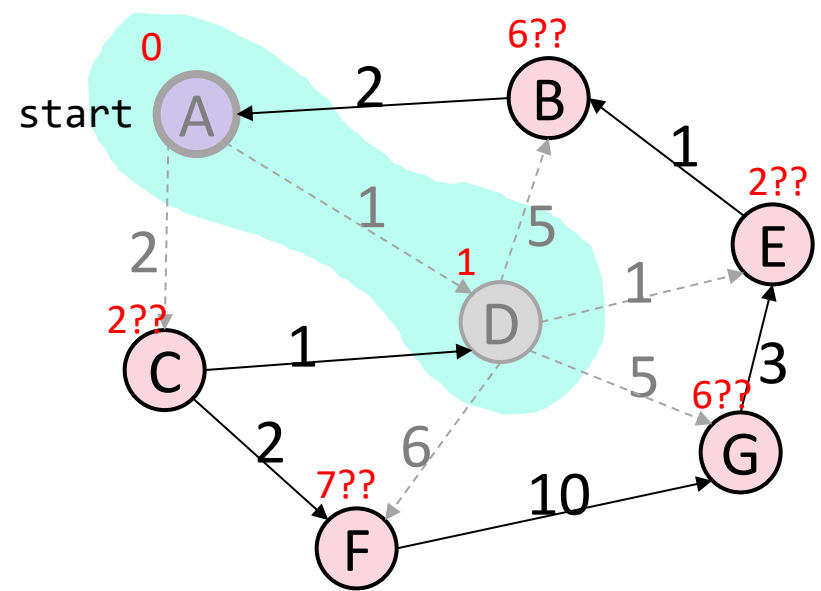


```
while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
    for each edge (u,v) to unknown v with w:
        if distTo.get(u) + w < distTo.get(v):
```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		∞	
C		≤ 2	A
D		≤ 1	A
E		∞	
F		∞	
G		∞	

Order Added to
Known Set:
A

Dijkstras Algorithmus: Übung



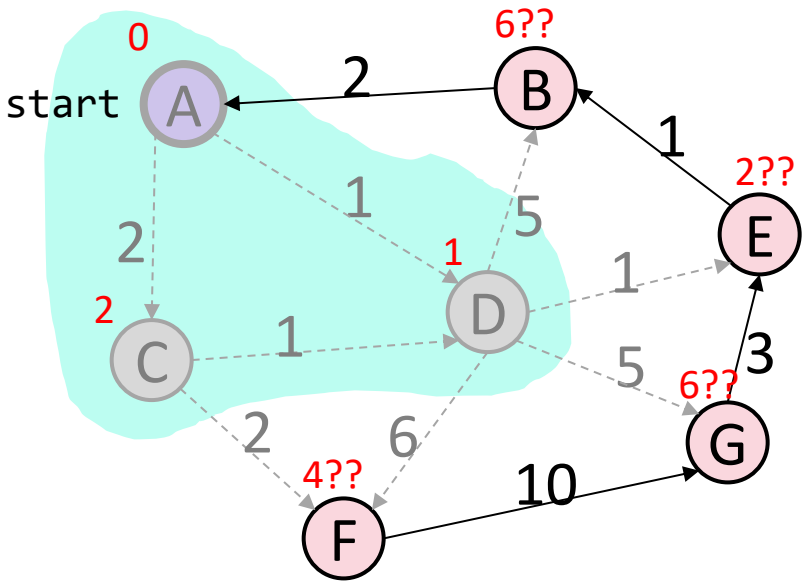
Order Added to
Known Set:
 A, D

```

while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
    for each edge (u,v) to unknown v with w:
        if distTo.get(u) + w < distTo.get(v):
    
```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 6	D
C		≤ 2	A
D	Y	1	A
E		≤ 2	D
F		≤ 7	D
G		≤ 6	D

Dijkstras Algorithmus: Übung



Order Added to
Known Set:
 A, D, C

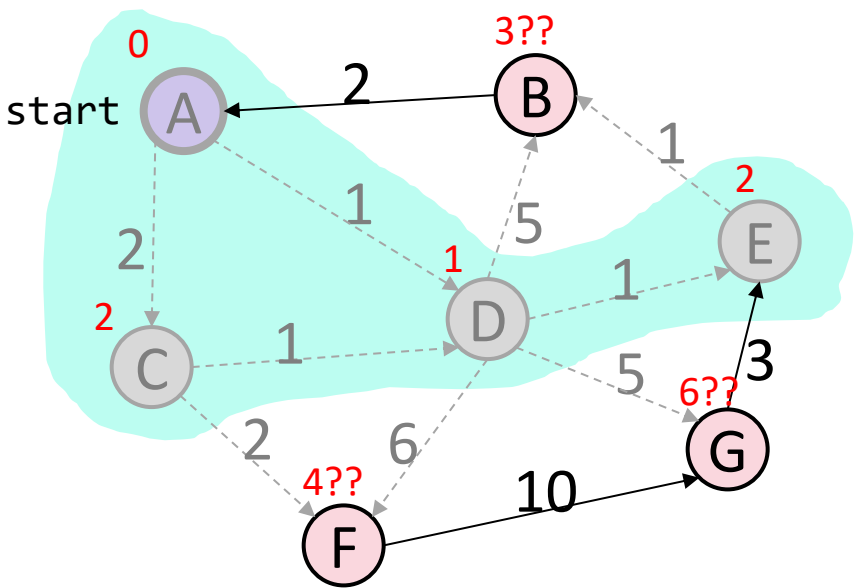
```

while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
    for each edge (u,v) to unknown v with w:
        if distTo.get(u) + w < distTo.get(v):

```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 6	D
C	Y	2	A
D	Y	1	A
E		≤ 2	D
F		≤ 4	C
G		≤ 6	D

Dijkstras Algorithmus: Übung



Order Added to
Known Set:
 A, D, C, E

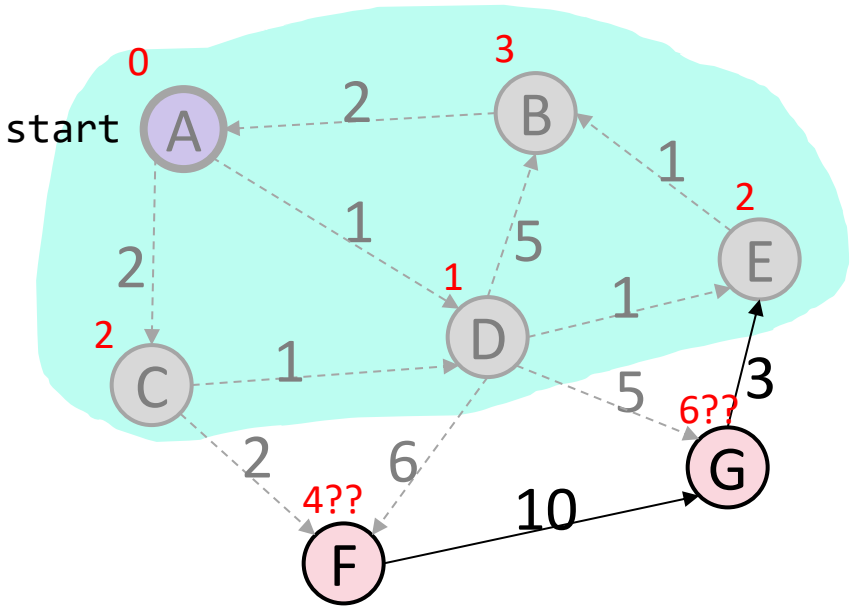
```

while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
    for each edge (u,v) to unknown v with w:
        if distTo.get(u) + w < distTo.get(v):

```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 3	E
C	Y	2	A
D	Y	1	A
E	Y	2	D
F		≤ 4	C
G		≤ 6	D

Dijkstras Algorithmus: Übung



Order Added to
Known Set:
 A, D, C, E, B

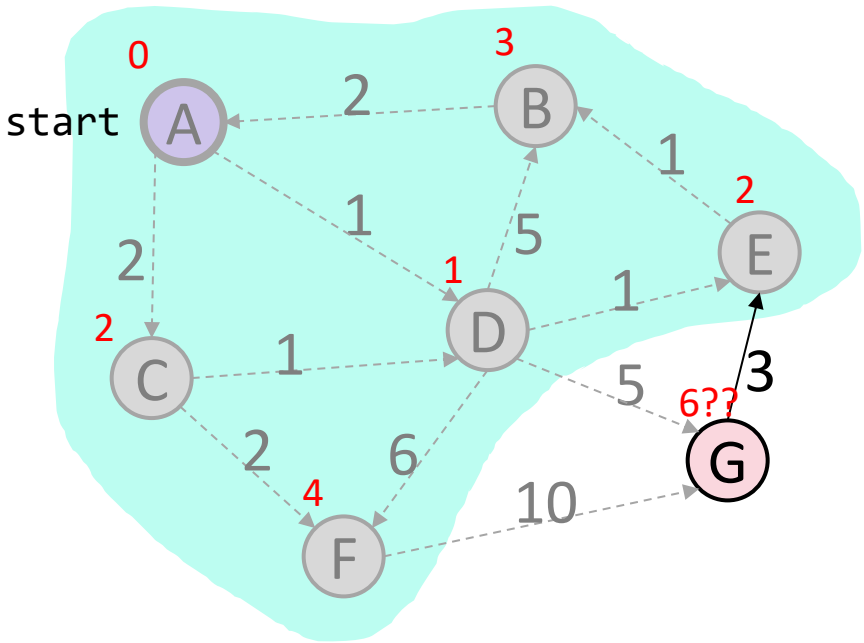
```

while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
    for each edge (u,v) to unknown v with w:
        if distTo.get(u) + w < distTo.get(v):

```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	3	E
C	Y	2	A
D	Y	1	A
E	Y	2	D
F		≤ 4	C
G		≤ 6	D

Dijkstras Algorithmus: Übung



Order Added to
Known Set:
 A, D, C, E, B, F

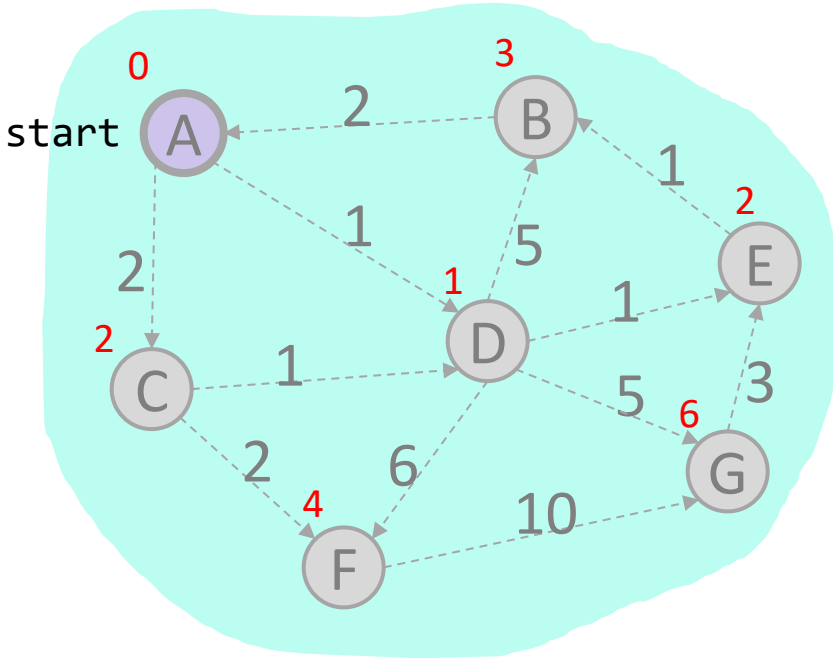
```

while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
    for each edge (u,v) to unknown v with w:
        if distTo.get(u) + w < distTo.get(v):

```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	3	E
C	Y	2	A
D	Y	1	A
E	Y	2	D
F	Y	4	C
G		≤ 6	D

Dijkstras Algorithmus: Übung



```

while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
    for each edge (u,v) to unknown v with w:
        if distTo.get(u) + w < distTo.get(v):

```

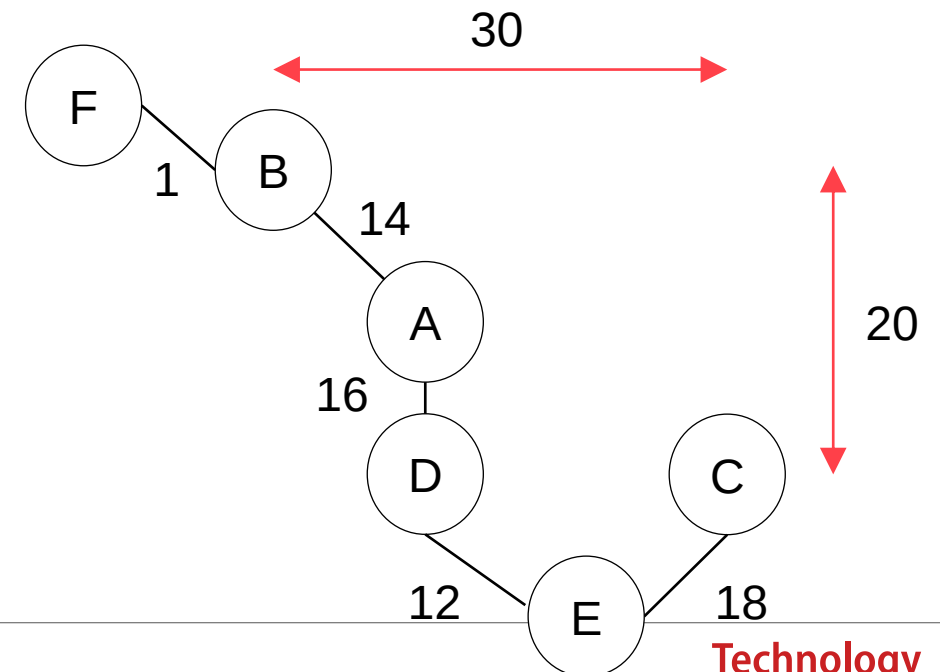
Order Added to
Known Set:
 A, D, C, E, B, F, G

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	3	E
C	Y	2	A
D	Y	1	A
E	Y	2	D
F	Y	4	C
G	Y	6	D

Bewegungsplanung

A* Algorithmus

- Ähnlich zu Dijkstra's Algorithmus, allerdings wird neben dem Start- auch der Zielknoten vorgegeben
 - Start: A, Ziel: C
 - Dijkstra würde B vor D besuchen, da B näher an A dran ist
- Für jeden Nachbarknoten wird eine Schätzfunktion berechnet, die schätzt wie weit der Knoten vom Zielknoten entfernt ist. Bspw.: euklidische Distanz
 - B: $36 \ (30^2 + 20^2)^{0.5}$
 - D: $20 \ (20^2 + 0^2)^{0.5}$
- Es wird der Nachbarknoten des aktuellen Knotens als nächstes betrachtet, über den laut Schätzung der kürzeste Pfad vom Start- zum Zielknoten geht
 - B: Distanz von A über B zu C: $14 + 36 = 50$
 - D: Distanz von A über D zu C: $16 + 20 = 36$
 - → A* geht zu D, da $36 < 50$



Bewegungsplanung

A* Algorithmus

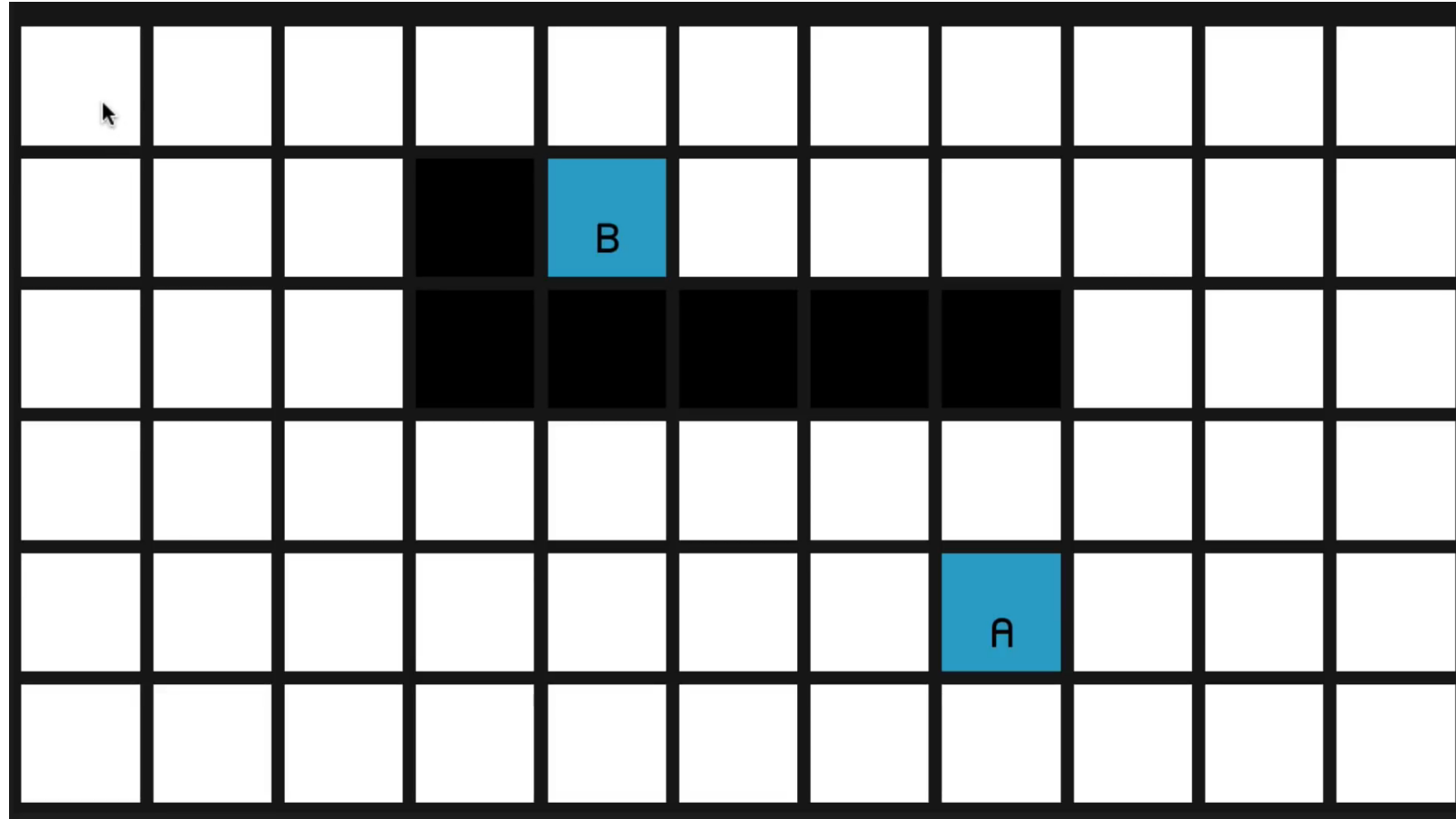
- Ähnlich zu Dijkstra's Algorithmus, allerdings wird neben dem Start- auch der Zielknoten vorgegeben
- Für jeden Knoten wird eine Schätzfunktion berechnet, die schätzt wie weit der Knoten vom Zielknoten entfernt ist
- Es wird der Nachbarknoten des aktuellen Knotens als nächstes betrachtet, über den laut Schätzung der kürzeste Pfad vom Start- zum Zielknoten geht

Beispiel:

- Wir wollen von Köln nach München
- Dijkstra würde auch Knoten untersuchen, die nördlich von Köln liegen
- A* berechnet/schätzt die Distanz von jedem Knoten zum Zielknoten (bspw. euklidische Abstand) und geht deshalb gleich südlich und ignoriert die Knoten nördlich von Köln

Bewegungsplanung

A* Algorithmus



Bewegungsplanung

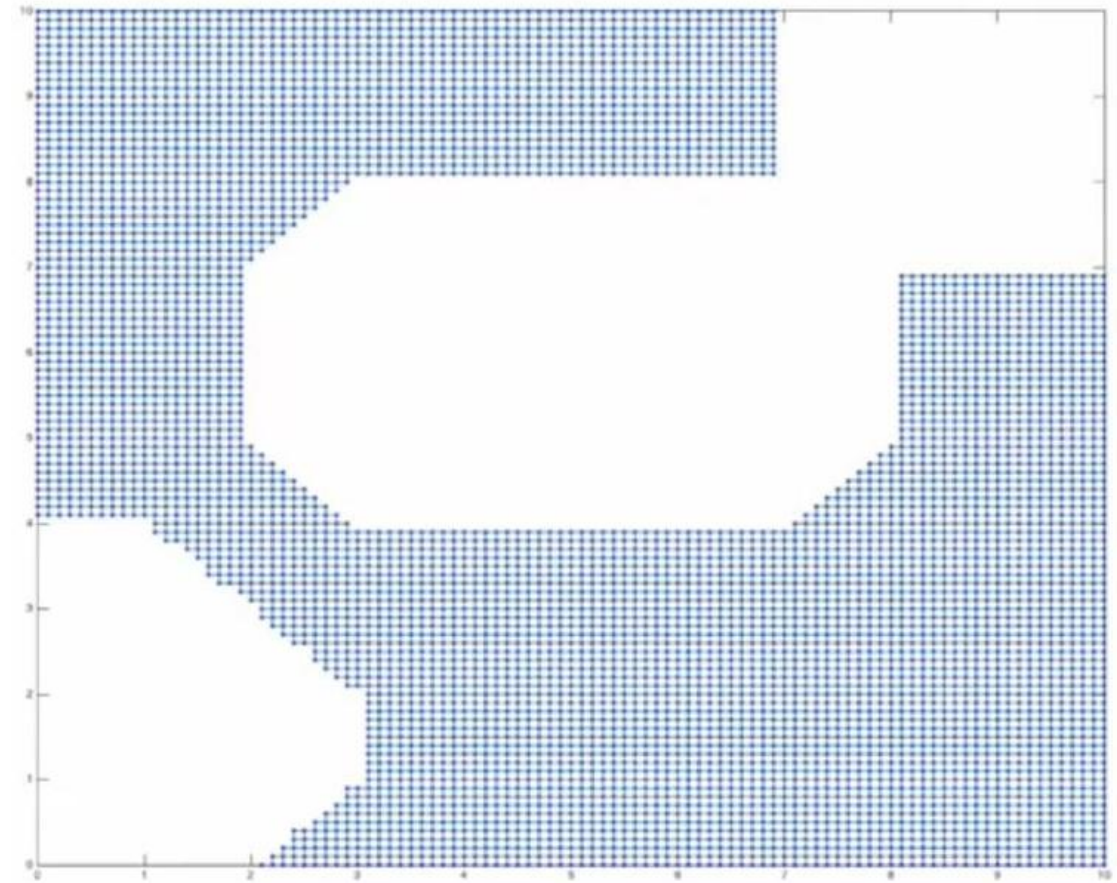
Erstellung des Graphen

Der Graph wird sehr groß, wenn wir versuchen den C_{free} -Space sehr detailliert zu approximieren (diskretisieren)

Nicht praktikabel für hochdimensionale C-Spaces

- Roboterarm mit 6 DOF -> 6-dim. C-Space
- DOF: Degrees of Freedom

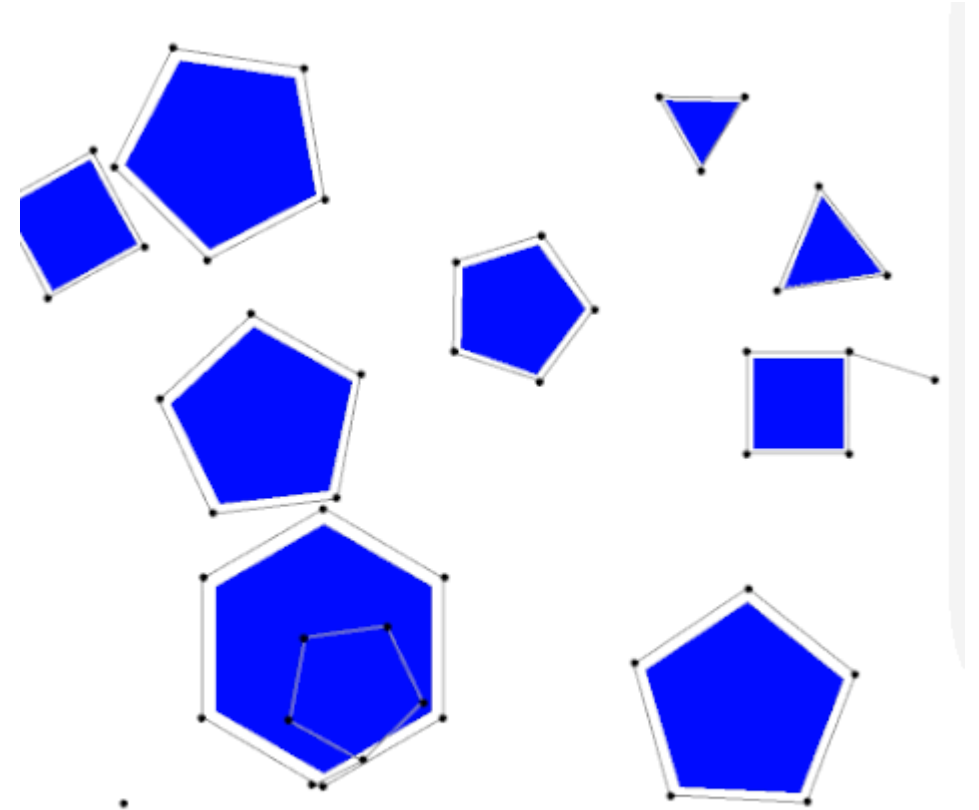
Wir benötigen bessere Methoden, um den Graph zu erstellen



Bewegungsplanung

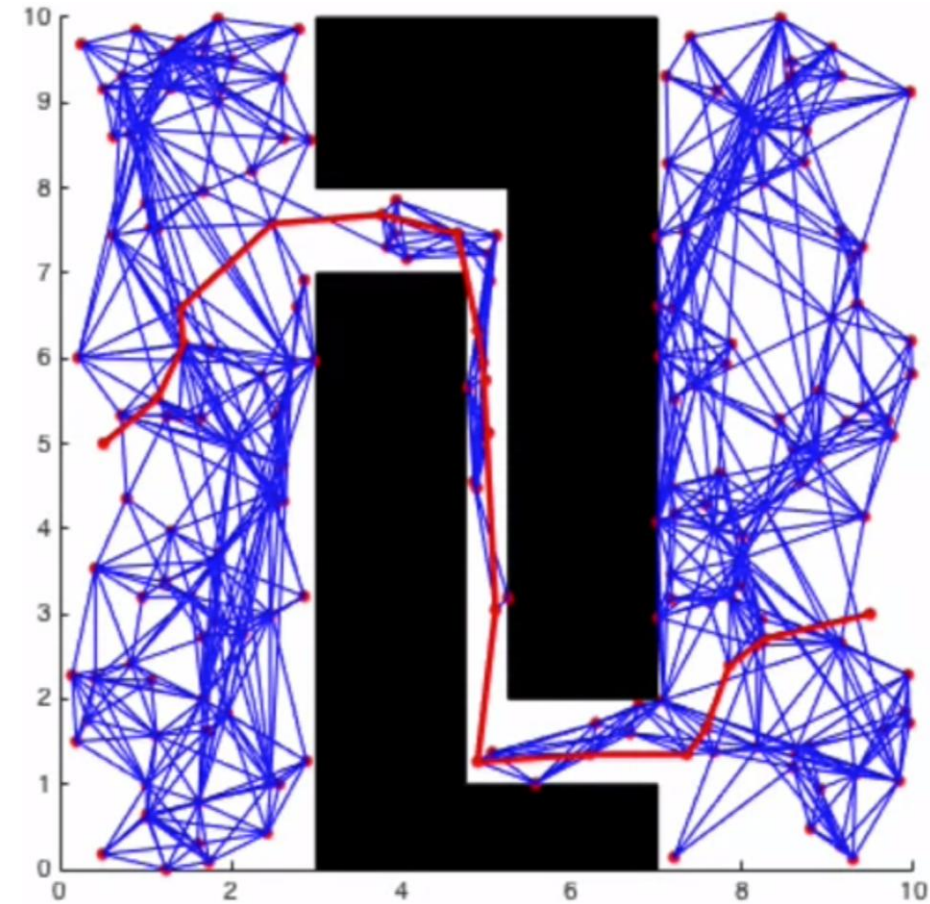
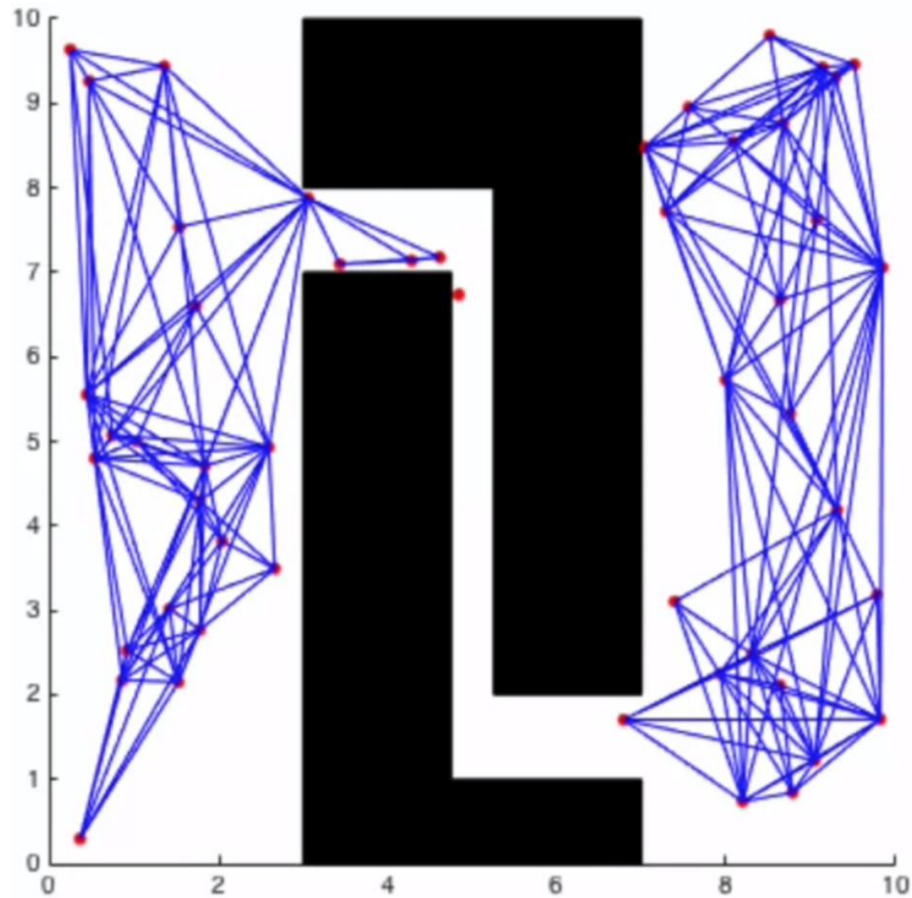
Probabilistic Roadmap

- Zufälliges Sampling von Knoten im C_{free} -Space und versuchen diese mit Nachbarknoten über Geraden zu verbinden
- Bild rechts: Hindernisse sind blau



Bewegungsplanung

Probabilistic Roadmap

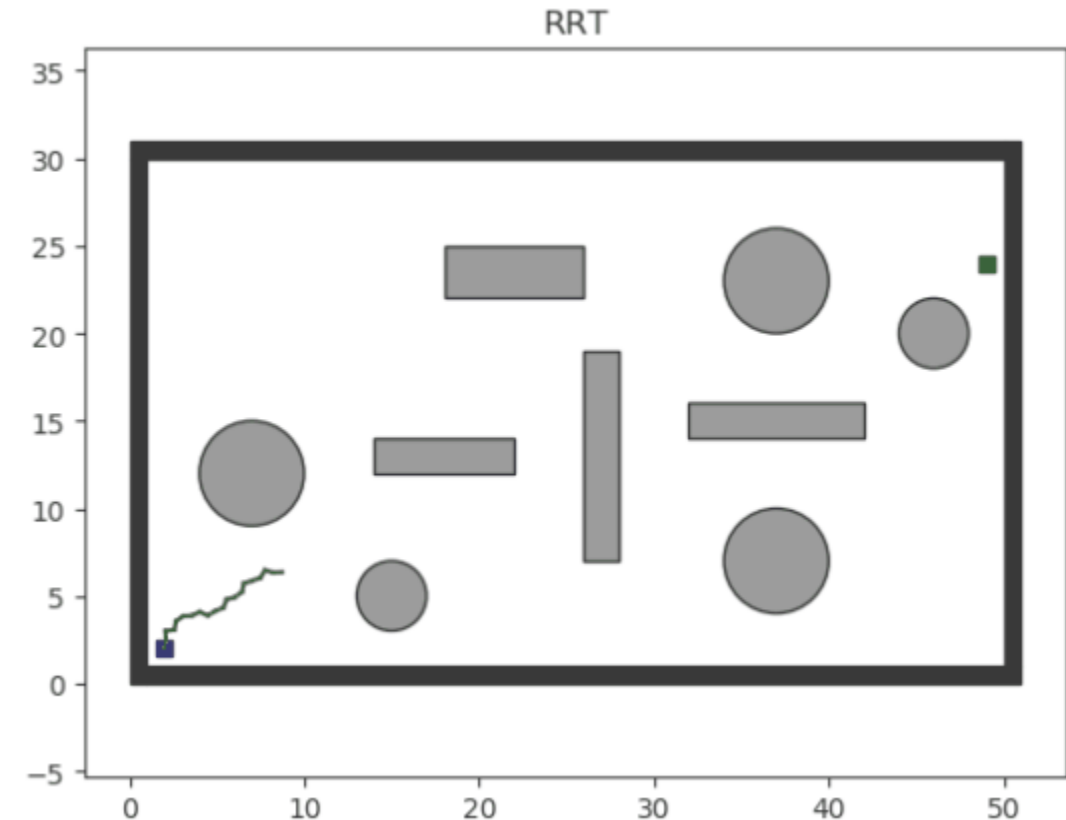


Bewegungsplanung

Rapidly Exploring Random Tree (RRT)

Erweiterung des A* Algorithmus um

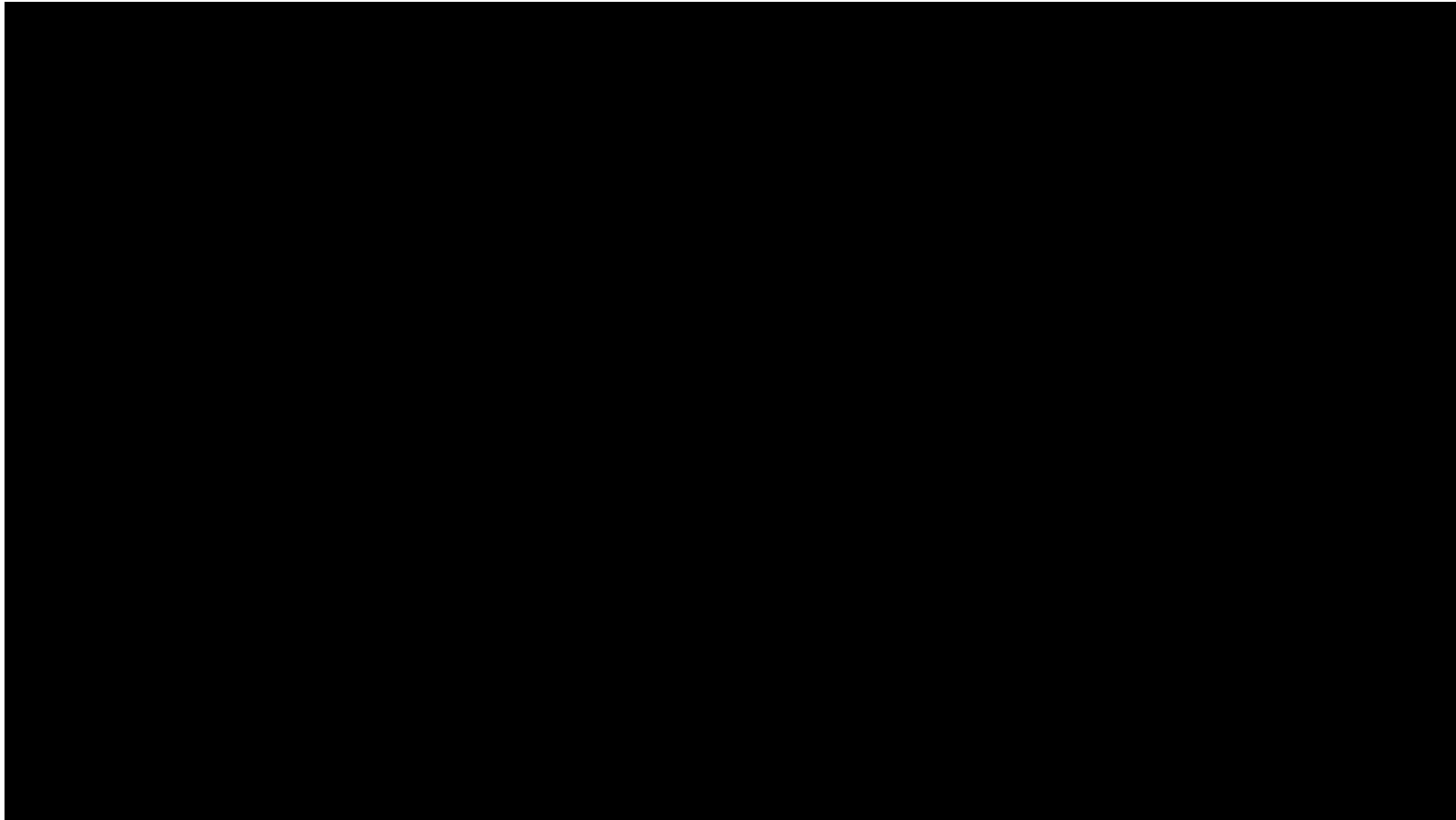
- Voronoi Bias
 - In einem Bereich mit wenig Knoten wird ein Knoten hinzugefügt
- Fähigkeit neue Knoten dem Graphen hinzuzufügen



<https://github.com/zhm-real/PathPlanning>

Bewegungsplanung

Rapidly Exploring Random Tree



Bewegungsplanung

Tools

Umfangreiche Bibliothek:

- The Open Motion Planning Library: “OMPL, the Open Motion Planning Library, consists of many state-of-the-art sampling-based motion planning algorithms.”
<https://ompl.kavrakilab.org/>
- ROS 2
 - MoveIt 2, <https://moveit.picknik.ai/main/index.html>

Übung – Planung

medium.com

Was für Planungen muss man im humanoiden Roboterfußball durchführen?

- Angenommen der Ball liegt oder rollt in der Nähe und wir wollen zu dem Ball laufen



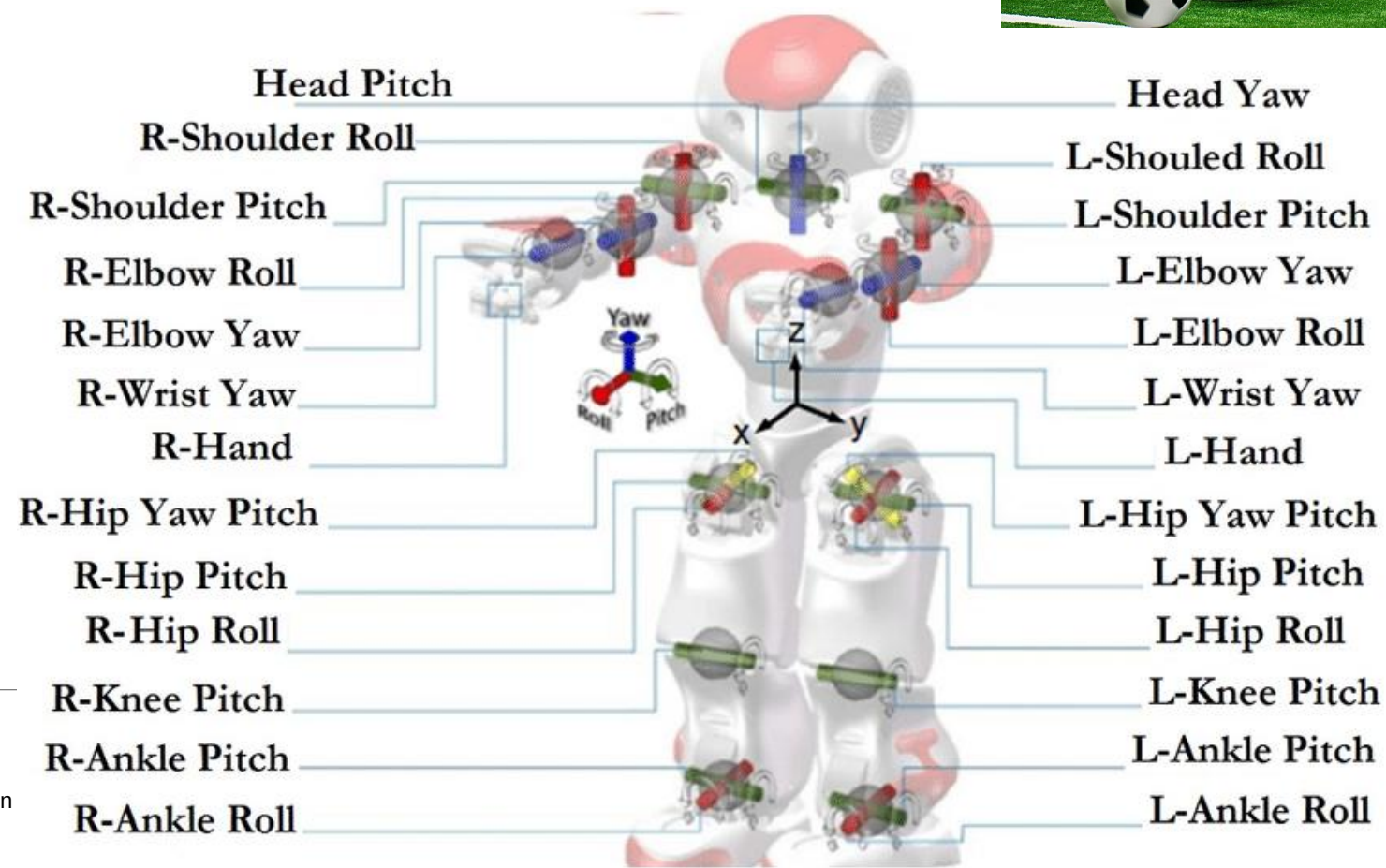
Übung – Planung

medium.com



Was für Planungen muss man im humanoiden Roboterfußball durchführen?

- Was schätzen Sie wie viele Freiheitsgrade der humanoide Roboter Nao hat?



Quelle des Bildes: https://www.researchgate.net/figure/NAO-humanoid-robot-v50-joints-and-base-frame-1_fig1_344611241

Inhalt

Prognose:

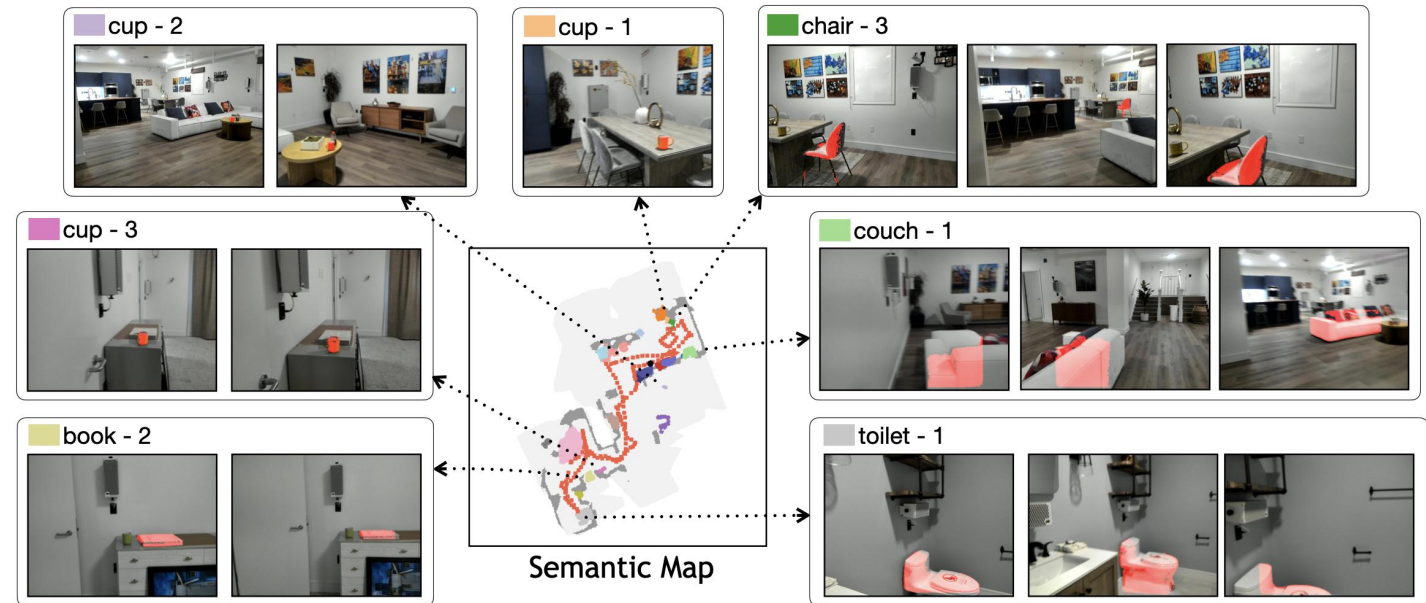
- Prognose mit Deep Learning Modellen

Planung:

- Bewegungsplanung
 - Configuration Space (C-Space)
 - Approximation des C_{free} -Space durch einen Graph
 - Algorithmus von Dijkstra und A* für Suche des kürzesten Pfades auf dem Graph
 - Probabilistic Roadmap und Rapidly Exploring Random Tree für bessere Graph Erstellung
- **Aufgabenplanung**
 - „Gehe zur großen weißen Couch“

Aufgabenplanung

- „Gehe zur großen weißen Couch“
- Nutzung der Semantischen Karte
 - Ermittlung der Koordination der „großen weißen Couch“
- Rest wie bei Bewegungsplanung



Aufgabenplanung: Komplexeres Beispiel

- Auftrag beim PC-Versandhändler:
 - Baue in folgenden PC die zusätzlichen Komponenten ein und bereite PC für Versand vor



Precision 3680 Tower[Core i9, 16GB, mit Grafikoptionen]

Artikelnummer 210-BLLP

Intel® Core™ i9 14900 der 14. Generation (36 MB Cache, 24 Cores, 32 Threads, 2,0 GHz bis 5,8 GHz, 65 W)...

zzgl. MwSt. 7%

+ 128 GB: 4 x 32 GB, DDR5, 3600 MT/s, non-ECC | Arbeitsspeicher

+ 2 TB, 7200 RPM, 3.5-inch, SATA, HDD | Storage

+ NVIDIA® RTX™ 5000 Ada Generation, 32GB GDDR6, 4DP | Basisoptionen

+ Intel 2.5GbE i226 FH PCIe Network Adapter (NIC) Card | Netzwerkkarte

+ 8x DVD+/-RW 9.5mm Optical Disk Drive | Optisches Laufwerk

+ 1 TB, M.2 2280, Gen 4 PCIe NVMe, SSD | 1st M.2 NVMe SSD



01.09.2026

Seite 57

Bildquelle: <https://hello-robot.com/stretch-3-product>

Aufgabenplanung: Komplexeres Beispiel

- Roboter versteht die Aufgabe und erstellt Plan:

Fähigkeit: Sprachverständnis und planendes Handeln

1. Welche Werkzeuge und Bauteile werden für diesen Vorgang in welcher Reihenfolge benötigt?

Fähigkeit: Verständnis über die Welt

2. Wo wird jede dieser Komponenten in der Werkstatt gelagert?

Fähigkeit: Erkennen der Umgebung über Kamera und Erstellung einer Karte

3. Fahre zu jedem potenziellen Aufbewahrungsort und suche dort nach der jeweiligen Komponente.

Fähigkeit: Autonome Navigation, Greifen von Objekten

4. Bringe die gefundenen Komponenten zu dem Arbeitsplatz.



Closed-Loop Open-Vocabulary Mobile Manipulation with GPT-4V



Sprache verstehen und Ausarbeiten von Plänen

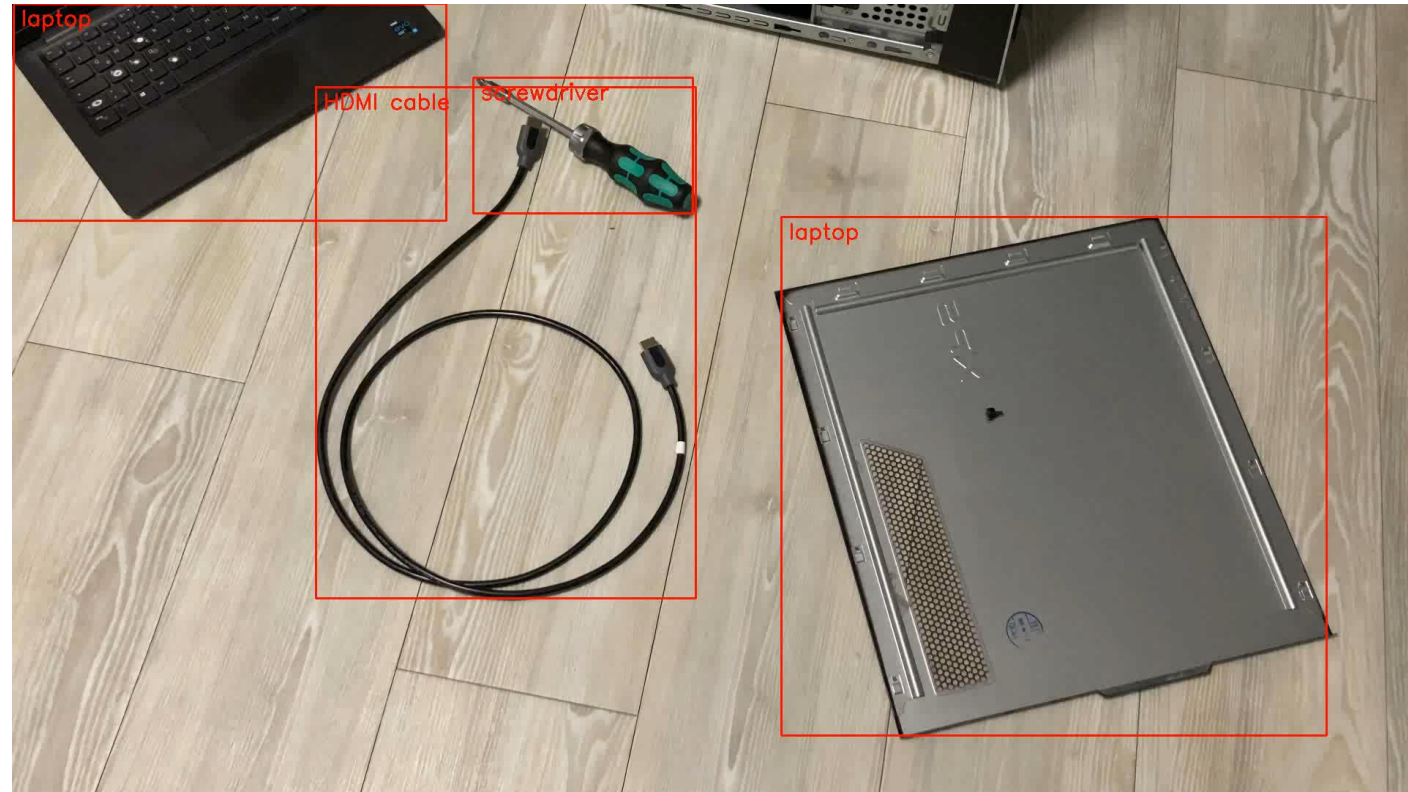
- Large Language Modelle wie GPT-4 ...
 - verstehen Sprache, generieren Text und haben ein Wissen über die Welt
 - können Pläne ausarbeiten
 - Aus einer Anweisung einen Schritt-für-Schritt Plan entwerfen
- „Could you give me some food? And pass me a cup of juice.“
 1. Explore global
 2. Navigate to table_0
 3. Explore local
 4. Observe bottles and cup
 5. Grasp bottle_1
 6. Grasp banana_0
- Plan hängt von der Umgebung ab (bspw. Objekte in der Umgebung)

Open-Vocabulary Objekt Detektion

- Man kann angeben welche Objekte erkannt werden sollen.

Hier:

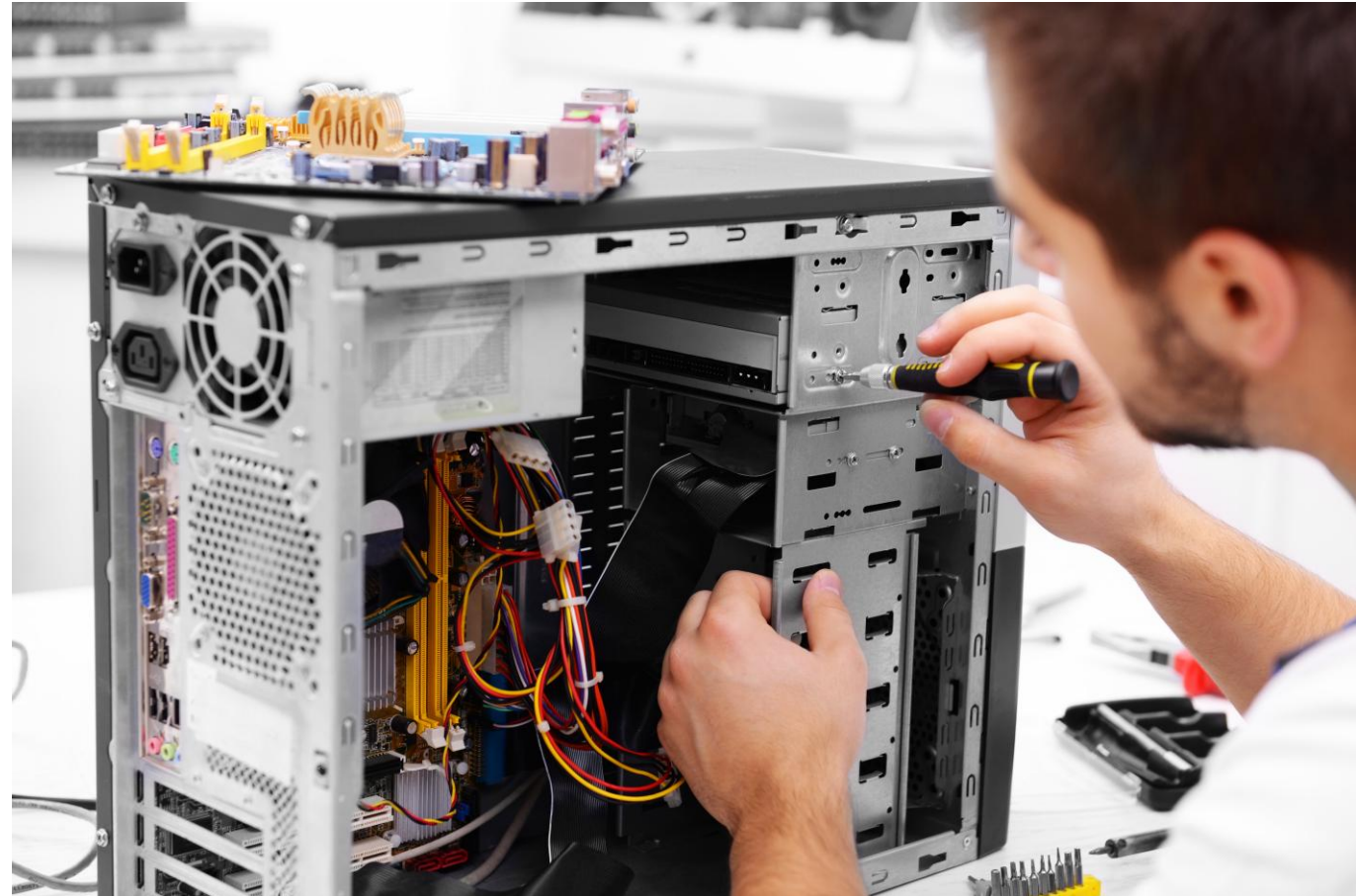
- computer,
- laptop,
- PC fan,
- HDMI cable,
- screwdriver,
- power supply unit
- motherboard



Multimodale Large Language Modelle: GPT-4o

GPT-4o erkennt folgende Objekte:

- PC-Gehäuse
- Motherboard
- Lüfter
- SATA-Kabel und Stromkabel
- Laufwerksschacht
- Schraubendreher
- Netzteilanschluss
- Person



Aufgabenplanung: Komplexeres Beispiel

- Roboter versteht die Aufgabe und erstellt Plan:

Fähigkeit: Sprachverständnis und planendes Handeln

1. Welche Werkzeuge und Bauteile werden für diesen Vorgang in welcher Reihenfolge benötigt?

Fähigkeit: Verständnis über die Welt

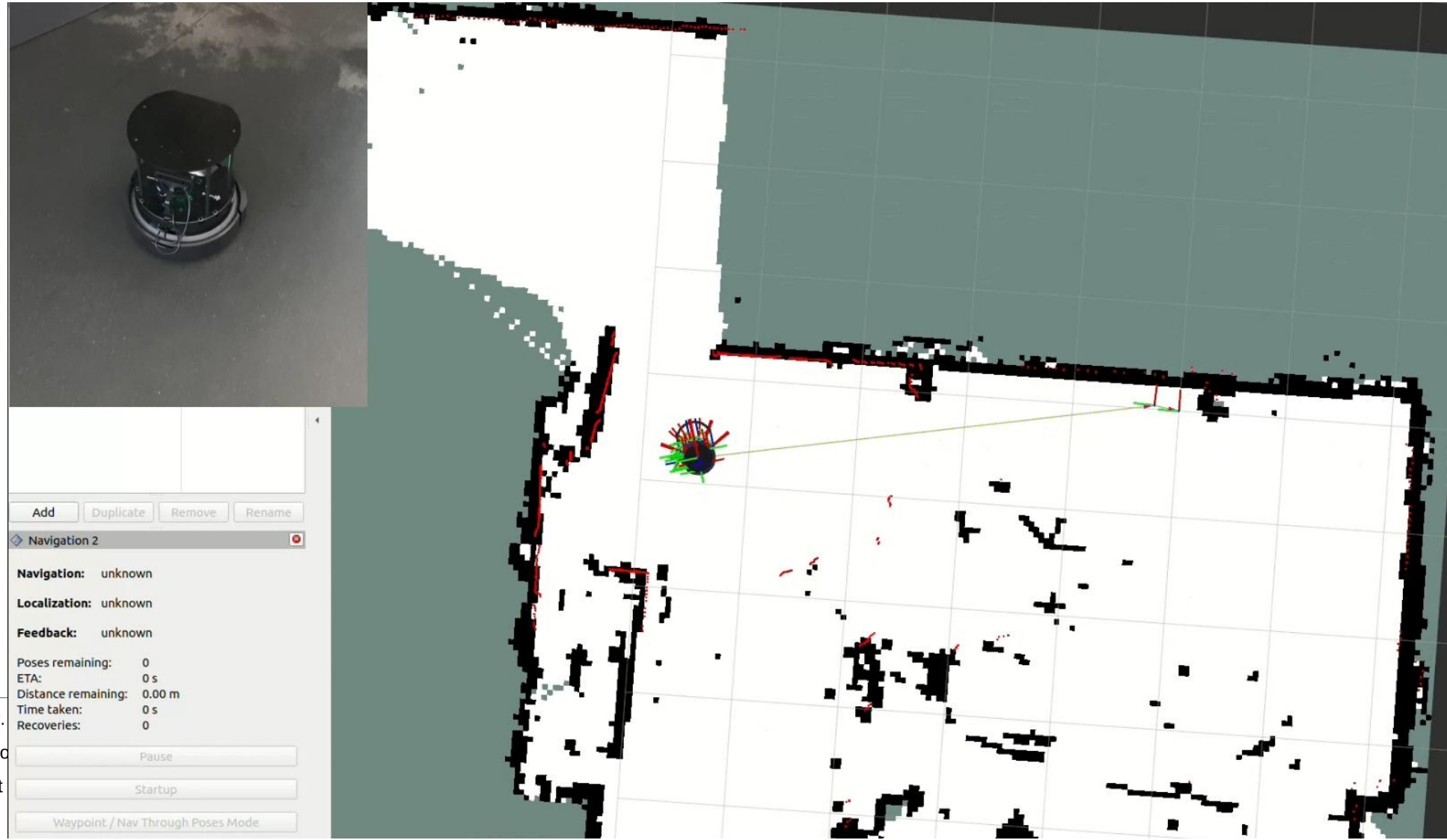
2. Wo wird jede dieser Komponenten in der Werkstatt gelagert?

Fähigkeit: Erkennen der Umgebung über Kamera und **Erstellung einer Karte**

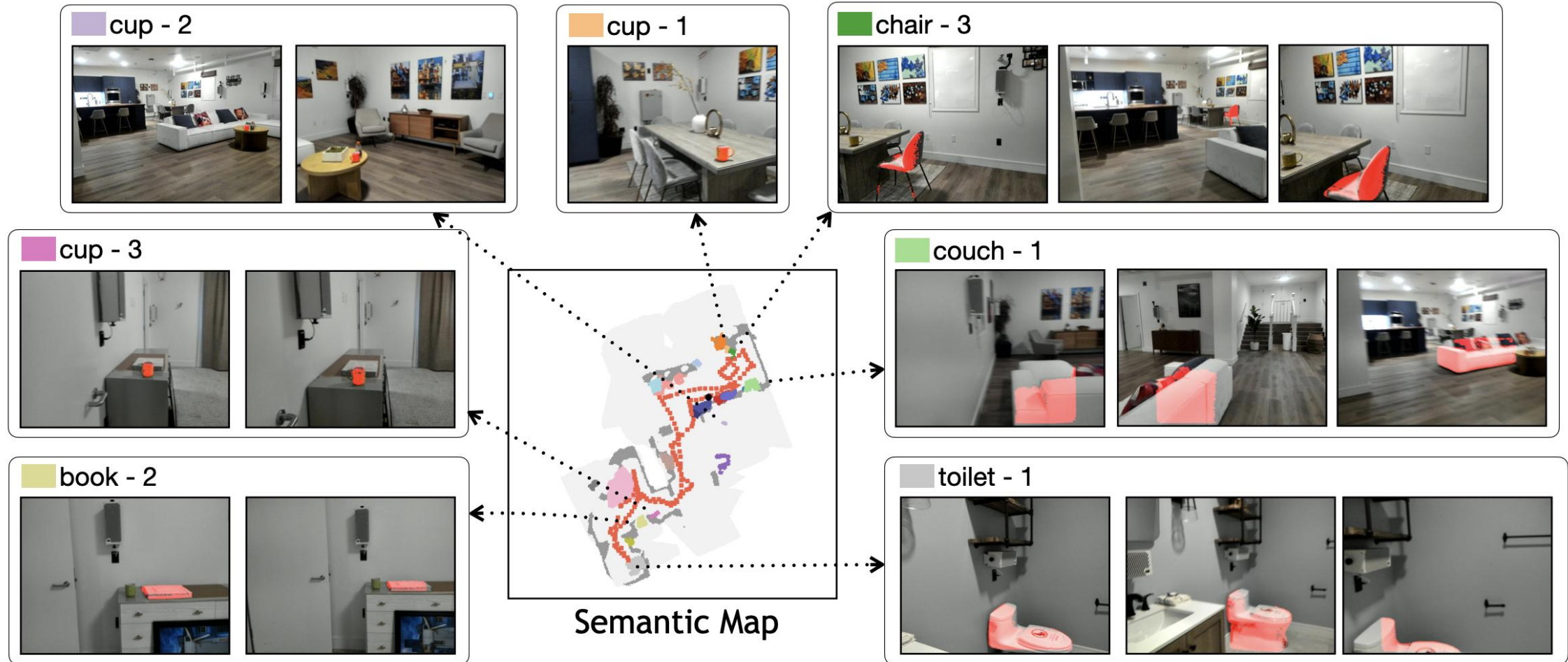


Erstellung einer Karte

Karte des Flures
vor Raum 1.242
erstellt durch
Turtlebot



Semantische Karte (Semantic Mapping)



Semantische Karte (Semantic Mapping)

Aufgabenplanung: Komplexeres Beispiel

- Roboter versteht die Aufgabe und erstellt Plan:

Fähigkeit: Sprachverständnis und planendes Handeln

1. Welche Werkzeuge und Bauteile werden für diesen Vorgang in welcher Reihenfolge benötigt?

Fähigkeit: Verständnis über die Welt

2. Wo wird jede dieser Komponenten in der Werkstatt gelagert?

Fähigkeit: Erkennen der Umgebung über Kamera und Erstellung einer Karte

3. Fahre zu jedem potenziellen Aufbewahrungsort und suche dort nach der jeweiligen Komponente.

Fähigkeit: Autonome Navigation, Greifen von Objekten

4. Bringe die gefundenen Komponenten zu dem Arbeitsplatz.



Lernziele

Die Studierenden können für eine Umgebung angeben welche Prognosen notwendig sind, indem sie die Umgebung durch Deep Learning Modelle modellieren und mit diesen Prognosen/Vorhersagen machen, um später Cyber-physische Systeme entwickeln zu können, die Veränderungen der Umgebung prognostizieren können.

Die Studierenden können mit dem Modell eines CPS Planungen durchführen, indem sie für einfache CPS den Configuration Space modellieren, diesen durch einen Graphen approximieren und Algorithmen zur Suche nach dem kürzesten Pfad darauf anwenden, um später Cyber-physische Systeme entwickeln zu können, die einen Plan machen können.

Workshop (20 Minuten)

Diskutieren Sie in Ihren Projektteams:

- Ist in Ihrem Projekt die Umgebung Ihres CPS dynamisch?
- Können Sie in der Umgebung etwas prognostizieren? Falls ja, wie gehen Sie vor?

- Wird Ihr CPS planen können/müssen?
- Falls ja,
 - Was wird geplant? Wie wird geplant?

Präsentation der Ergebnisse:

- Jedes Team präsentiert (1-2 Minuten)

Nächste Vorlesung und Fragen

- ...

- Fragen?